

Tutor Mx System

Gitea Sprint Handbook and Review Guide

Beginner-friendly Sprint 1-4 issue, testing, handover and deployment guide
for six Members

CURRENT STACK

React + Vite frontend | Auth0 | handwritten Node.js + Express API | Prisma | Neon PostgreSQL | Gitea main | GitHub
deployment mirror | Codecov | Cloudflare Pages | Render API

COMS3011A Project 1 | 2026

Fully updated 11 September 2026

AI Declaration: This edition was updated with ChatGPT [GPT-5.6 Sol] from the supplied handbook, the current integrated Tutor Mx source and the team's recorded workflow. The team remains responsible for checking requirements, commands and evidence before submission.

UPDATED THROUGH 16 SEPTEMBER 2026

Sprint 1-2 history retained; post-Sprint 2 stabilisation and Sprint 3-4 roadmap revised.

What changed in this edition

- Gitea main is the official shared code. The old handbook instructions that told Members to target a develop branch are removed.
- All six Sprint 1 Member contributions are treated as the current integrated baseline. Sprint 2-4 work builds on that baseline instead of recreating it.
- New users choose Student, Tutor or Organiser before Auth0 sign-up. Express performs server-side onboarding and creates/links the matching Neon Profile automatically.
- Existing users sign in once and are routed directly to their saved role workspace. Normal sign-in does not ask them to choose a role again.
- Organiser Tutor Management now lists self-registered Tutor accounts. Organisers search by name/email and Edit the database-backed profile; they never type an Auth0 subject/User ID.
- The exact current repository test gate (npm run check), coverage commands and local browser verification steps are documented on pages 7-9.
- The lecturer-runner rule is updated: two shared Gitea Action runners are now available to all groups; queued jobs are normal, local tests remain required, and completed CI evidence must be kept.
- The deployment path is clarified: approved Gitea main -> GitHub deployment mirror -> fresh Vite build -> manual Cloudflare Production upload for the frontend.

CURRENT SPRINT 1 BASELINE

Members 1-6 are integrated in the current main-ready source. The Organiser/Tutor features use real API routes where implemented; Student overflow/volunteer remains the deliberate Sprint 1 mock boundary until Sprint 2 connects the real Student API. The Sprint 1 Save allocation control is still a planned placeholder until the allocation-write endpoint is completed.

1 SEPTEMBER RUNNER UPDATE

Shared runners: sdp-runner-1 and sdp-runner-2. They are shared across groups, so jobs may queue and then auto-process. Each runner is limited to 4 GB memory and currently a 3-hour timeout. A private group runner is optional, not required.

How the team works now

Use this method in Sprint 1, 2, 3 and 4

NO DEVELOP BRANCH

Gitea main is the official shared branch. Members do normal feature work on their own branch. The Integration Lead reviews/integrates approved work into main.

Step	What to do
1. Refresh	git switch main -> git pull origin main
2. Create/use your branch	git switch -c sX/mY/short-name (or switch to your existing issue branch)
3. Work locally	Make only the changes for your issue. Keep secrets out of files/screenshots.
4. Test	Follow page 7. Then run the browser/local verification on page 8.
5. Commit your branch	Commit focused work on your branch. Do not commit directly to main.
6. Push + hand over	Push the branch to Gitea and tell the group: issue ID, branch, tests run, known limits, AI attribution.
7. Review + integrate	Reviewer checks the issue. Integration Lead integrates approved work into main and reruns the affected regression/full gate.
8. Refresh again	After main changes, everyone pulls the latest main before starting/resuming later work.

INTEGRATION LEAD EXCEPTION

When the Integration Lead performs integration/repository-maintenance work themselves, reviewed/tested changes may be committed through their integration workflow to main. This does not turn main into a normal Member working branch.

NEW SHARED-RUNNER STEP AFTER PUSH

After a Member pushes the branch, confirm the Gitea Actions workflow starts. If it is queued, wait for a shared runner to become free. Keep the completed run link/status with the handover evidence. The reviewer and Integration Lead check the CI result as part of review; local page 7-8 testing is still required.

SPRINT 3-4 CURRENT METHOD - ONE BRANCH PER MEMBER

For Sprint 3 and Sprint 4, each Member uses one branch for all of that Member's sprint work. Member 1 branches from current main, finishes all assigned work, pushes and hands over. The Integration Lead reviews/tests and merges that branch to main. Only then does Member 2 pull the updated main and create their one branch. Repeat through Member 6. Do not branch from the previous Member's feature branch and do not create separate branches for individual cards.

Architecture and automatic role onboarding

Layer	Beginner meaning
1. Browser	Loads Tutor Mx. New user chooses Student / Tutor / Organiser.
2. Cloudflare Pages	Serves the React + Vite frontend. Only public VITE_* configuration belongs here.
3. Auth0	Universal Login handles Google or email/password identity and email verification.
4. React onboarding	First-time account calls POST /api/auth/onboarding with the selected role + bearer token.
5. Express on Render	Verifies Auth0 identity, applies role policy and uses server-only Auth0 Management credentials when needed.
6. Prisma + Neon	Creates/reads the matching Profile and application data. Browser code never connects directly to Neon.
7. Later sign-ins	React calls /api/me and opens the saved role workspace automatically.
ONE ROLE PER TUTOR MX ACCOUNT The selected role is treated as one-time. Use separate test accounts/emails for Student, Tutor and Organiser. Do not use normal onboarding to switch an established account between roles.	

SECRETS RULE

DATABASE_URL and Auth0 Management API client secrets stay on the backend host. They must never be placed in VITE_* variables, screenshots, issues, chat, or committed files.

Very beginner guide: create an account or sign in

Action	Expected path
Create Student	Choose Student -> Create Student account -> Auth0 -> server onboarding -> /student
Create Tutor	Choose Tutor -> Create Tutor account -> Auth0 -> server onboarding -> /tutor
Create Organiser	Choose Organiser -> Create Organiser account -> Auth0 -> server onboarding -> /organiser
Existing account	Click Sign in -> Auth0 -> /api/me -> correct saved workspace. No second role choice.
Switch test role	Sign out first, then use the separate account created for the other role.
EMAIL VERIFICATION If Auth0 requires verification for an email/password account, verify the email and return to Tutor Mx. An unverified account must not be silently assigned a role.	

NORMAL SETUP IS AUTOMATIC

Do not manually create Neon Profile rows or manually assign individual Tutor Mx roles in Auth0 for ordinary new-user sign-up. Manual admin intervention is troubleshooting, not the normal Member workflow.

Organiser Tutor Management - current professional method

WHAT THE ORGANISER SEES

Search for a tutor by name or email. Select Edit to manage the tutor's details and weekly-hour limit.

- Tutor accounts appear after the Tutor signs up through the normal self-service Tutor flow.
- The Organiser does not create the Tutor identity and does not copy an Auth0 User ID/subject.
- The UI uses the tutor's internal database record automatically when the Organiser edits details.
- Marks and busy times are managed from the selected Tutor record.
- Technical explanations about Auth0/Neon are documentation concerns; they should not clutter the normal organiser interface.

SPRINT PLANNING CONSEQUENCE

Sprint 2 must not rebuild a manual tutor-create/Auth0-subject form. Sprint 2 hardens the registered-tutor management flow and completes allocation writes/approvals.

How to run the tests in VS Code / PowerShell

Every Member uses this page before handing work over

FAST RULE

From the repository root, npm run check is the current full local release-style gate. It runs Prisma generation, lint, backend type-checking, backend + frontend tests, production builds and the repository security check.

```
npm ci
npm ci --prefix frontend
npm run generate
npm run check
npm run coverage
```

Command	What it proves
npm run check	Full gate: generate -> lint -> typecheck -> tests -> build -> security check.
npm test	Backend + frontend automated tests.
npm run test:backend	Backend Vitest suite only.
npm run test:frontend	Frontend Vitest suite only.
npm run coverage	Backend + frontend coverage reports.
npm run lint	Backend + frontend lint.
npm run typecheck	Backend TypeScript check.
npm run build	Production backend + frontend builds.
npm run security:check	Tracked-file secret / repository safety check.

WHAT COUNTS AS PASSING

The command must exit successfully. Warnings are not the same as failed tests, but new warnings caused by your change should still be investigated and mentioned in the handover. Never hide a failed command.

AFTER THE LOCAL GATE PASSES

Commit and push the Member branch to Gitea, then confirm its Actions workflow runs on the shared lecturer runners. A queued job is not a failure; wait for it to auto-process. Save the run link/logs and Codecov result with the handover evidence.

How to check your work locally before the Integration Lead touches main

PURPOSE

This is the beginner smoke test Members run on their own branch. It answers: does my change actually work in the app before I tell the Integration Lead it is ready?

```
git switch main
git pull origin main
git switch <your-issue-branch>

# first clean setup / when dependencies changed
npm ci
npm ci --prefix frontend
npm run generate

# start frontend + backend together
npm run dev
```

- [] Open <http://localhost:5173>. The API normally runs at <http://localhost:3000>.
- [] Use the role/account relevant to your issue. For role testing, use separate Student, Tutor and Organiser test accounts.
- [] Repeat the acceptance path yourself: success case, invalid/failure case, and one relevant empty/permission case.
- [] Refresh the page and retry the action. Check that loading, error and success states remain usable.
- [] For protected flows, confirm another role cannot perform the action just by typing a URL or changing a request.
- [] Stop the dev servers with Ctrl+C. Then run `npm run check` and `npm run coverage`.
- [] Run `git status --short` and confirm only the files you intended to change are listed.

HANDOVER EVIDENCE

Send the issue ID + branch name + local commands/results + screenshot/API example where useful + known limitation. After pushing, also send the Gitea Actions run link/status and Codecov result once the shared runner finishes. Do not send secrets or production personal data.

AFTER PUSHING TO GITEA

Queued = normal on the shared runners; wait. Failed = investigate and rerun. Genuine runner/infrastructure outage = record the blocker plus local evidence and retry/contact staff. Never report CI as green unless the workflow actually completed successfully.

Lecturer update: shared Gitea Actions runners and CI evidence

WHAT CHANGED ON 1 SEPTEMBER 2026

The lecturer has set up two tested shared runners, sdp-runner-1 and sdp-runner-2, for all groups. Because they are shared, a job may sit in the queue and then auto-process when a runner becomes free. Local testing remains required.

WHAT WE MUST DO

- Before handover, run the page 7 automated checks and page 8 local acceptance/smoke checks.
- Push the Member branch to Gitea and let the Actions workflow run on the shared runners.
- If the job is queued, wait; queueing is expected when another group is using the runners.
- When the job completes, keep the run link/logs and Codecov result as CI evidence.
- If the workflow fails, investigate the code/configuration and rerun it. If the runner infrastructure genuinely remains unavailable, attach local evidence, record the blocker honestly and contact staff for persistent/complex runner issues.
- Do not weaken/remove tests and do not claim CI or Codecov is green when the workflow did not complete.
- A private runner is optional. If the group registers one, use a descriptive name such as <group-name>-runner-<id>; the lecturer may remove private runners for admin/security reasons.

RUNNER LIMITS AND QUEUE RULE

Shared runners are limited to 4 GB memory and currently time out after 3 hours; the lecturer may reduce the timeout to 30 minutes later. Keep workflows reasonably simple. Queued is not failed - wait for auto-processing.

Member handover -> Integration Lead -> Gitea main -> GitHub mirror

```
git status
git add <only-your-files>
git diff --cached --check
git commit -m "<clear issue-focused message>"
git push -u origin <your-issue-branch>
```

- Member: do not merge your own normal feature branch into main. Run local checks first, commit, and push the issue branch.
- After push: confirm Gitea Actions starts. If queued, wait for a shared runner; once complete, attach the run link/status and Codecov result to the handover.
- Reviewer: reproduce the issue acceptance checks, inspect changed files, and check both local evidence and the completed CI result.
- Integration Lead: pull latest main, integrate approved work, resolve conflicts with the author, rerun npm run check (plus coverage/database checks when relevant), then push Gitea main and verify the main-branch Actions result if the workflow triggers there.
- Deployment mirror: only after Gitea main is approved, sync that source to the GitHub deployment repository. GitHub is not the team source of truth.

BEFORE EVERY LATER TASK

After an integration changes main, pull main again before continuing Sprint 2/3/4 work. This prevents members from building on an old snapshot.

SPRINT 3-4 HANDOVER RULE

After each Member finishes their single Sprint 3/4 branch, the Integration Lead reviews/tests and merges it to Gitea main. The next Member must pull that updated main before creating their own single branch. This repeats Member 1 -> 2 -> 3 -> 4 -> 5 -> 6.

Manual Cloudflare Production deployment after approved integration

FRONTEND RELEASE PATH

Gitea main -> GitHub deployment mirror -> fresh local Vite build -> Cloudflare Pages Production upload.

```
git switch main
git pull origin main
cd frontend
Remove-Item -Recurse -Force dist -ErrorAction SilentlyContinue
npm run build
```

- [] Open Cloudflare Dashboard -> Workers & Pages -> tutor-mx -> Create deployment.
- [] Choose Production.
- [] Upload the newly generated frontend/dist folder.
- [] Confirm all files upload successfully, then select Save and deploy.
- [] Open tutor-mx.pages.dev and hard refresh (Ctrl + Shift + R).
- [] Run the affected role smoke test on the live site.
- [] Backend/API changes must also be deployed to the hosted Express service using the team's approved deployment mirror/host workflow.

VITE RULE

VITE_* values are embedded at build time. If production frontend environment values change, rebuild dist before uploading it.

AI policy and Global Definition of Done

AI policy

- [] Remove secrets, tokens and personal data before using an AI tool.
- [] Read, understand and test AI-generated code before committing it.
- [] If a commit contains AI-generated code, use the course-required Assisted-by: <tool>[<model>] attribution with the actual tool/model.
- [] Keep README/document AI declarations current. Never invent test evidence, stakeholder evidence or tool/model names.

Global Definition of Done - applies to every issue in Sprint 1-4

- [] The issue-specific acceptance checks pass.
- [] Page 8 local browser/behavior check is completed for the affected flow.
- [] Page 7 automated test gate is run; relevant coverage is produced.
- [] Follow page 9: wait for queued shared-runner CI, keep completed run evidence, and document genuine infrastructure blockers honestly.
- [] Only intended files are committed on the Member branch; no secrets/personal data are added.
- [] The branch is pushed to Gitea and the assigned reviewer/Integration Lead is told it is ready.
- [] The Member does not merge normal feature work to main. Integration Lead integrates approved work and reruns regression.
- [] AI attribution/declarations are correct.

Sprint 1: plan, order and handovers

3-25 August 2026 | Integrated foundation, public demo and one secure vertical path

SPRINT 1 IS THE BASELINE

Current main already contains the integrated Member 1-6 Sprint 1 foundation. Use these issue cards for traceability, regression and follow-up fixes; do not rebuild working foundations.

- Keep the automatic Auth0 -> Express onboarding -> Neon Profile method as the normal account path.
- Keep Organiser Tutor Management registered-tutor based; no Auth0 subject field.
- Keep Tutor routes own-identity based; no browser tutor-ID ownership.
- Keep Student overflow/volunteer on the Sprint 1 mock boundary until Sprint 2 real endpoints are integrated.
- Keep Save allocation as the Sprint 1 placeholder until Sprint 2 allocation-write work is ready.

Member	Sprint ownership
Member 1	Authentication, role security, notifications/search and shared release checks
Member 2	Organiser interface, allocation board, approvals, reports and scenarios
Member 3	Tutor dashboard, availability, timesheets, excuses, imports and swaps
Member 4	Student overflow plus shared responsive and accessible interface
Member 5	Handwritten Express API, validation, business rules, security and API tests
Member 6	Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment

EVERY MEMBER

Before handover: use page 7 for automated tests, page 8 for local browser verification, page 9 for the lecturer-runner rule, and page 12 for the Global Definition of Done.

Sprint 1 - Member 1

Authentication, role security, notifications/search and shared release checks

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M1-1 | Self-service Auth0 sign-up, role onboarding and direct routing

Branch: s1/m1/auth-entry | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Goal: As a new or returning user, I want Tutor Mx to link my authenticated identity to exactly one role and open the correct workspace without manual setup.

Acceptance checks

- A new user chooses Student, Tutor or Organiser before Auth0 sign-up; the server creates the matching Tutor Mx role/Profile automatically.
- An existing linked user signs in once and is routed directly to /student, /tutor or /organiser without choosing a role again.
- An unverified email or onboarding conflict is handled safely; passwords and Management API credentials never enter Tutor Mx browser code.

Member tasks

- [] Maintain the role-choice entry screen and Auth0 Universal Login flow.
- [] Use POST /api/auth/onboarding for first-time linking and /api/me for later sign-ins.
- [] Keep sign-out available from all three workspaces and cover role-routing failures with tests.

Handoff: Give Member 5 the bearer-token shape and give Members 2-4 the signed-in user/role interface.

S1-M1-2 | Account settings, password reset, deletion and route protection

Branch: s1/m1/auth-lifecycle | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: S1-M1-1 and role names Organiser/Tutor/Student

Goal: As a Tutor Mx user, I want safe account recovery and protected role navigation so that I can manage access without exposing private data.

Acceptance checks

- Password-reset requests use a safe non-enumerating response and Auth0 owns the actual password workflow.
- Account deletion requires confirmation and the privileged Auth0/database deletion runs only on the backend.
- Wrong-role or unlinked access is blocked even if a user manually types a protected URL.

Member tasks

- [] Maintain Account Settings reset/deletion actions and confirmation states.
- [] Keep route guards aligned with backend role/ownership checks.
- [] Test cancel/confirm, wrong-role, missing-profile and signed-out paths.

Handoff: Give Member 5 the role matrix and give the Project Coordinator screenshots/test evidence for the security guide.

S1-M1-3 | Authentication contract and permission tests

Branch: s1/m1/auth-contract-tests | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: S1-M1-1, S1-M1-2 and S1-M5-2

Goal: As the team, we want one tested bearer-token and role contract so that every feature uses the same secure identity rules.

Acceptance checks

- Protected routes return 401 for missing/invalid tokens and 403 for valid users with the wrong role.
- A permitted token reaches the handler only when its Tutor Mx role matches the linked Neon Profile.
- Fresh first-login onboarding is handled safely even when the first token predates the newly assigned role.

Member tasks

- [] Maintain the shared bearer-token request helper.
- [] Keep backend Auth0 issuer/audience/JWKS/expiry checks covered.
- [] Add contract tests whenever a new protected route is introduced.

Handoff: Member 5 consumes the token helper/fixtures; Members 2-4 use the request wrapper rather than copying auth code.

Sprint 1 - Member 2

Organiser interface, allocation board, approvals, reports and scenarios

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M2-1 | Organiser workspace and allocation-board foundation

Branch: s1/m2/organiser-board-shell | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: Shared navigation from S1-M4-2; mock candidate contract from

Goal: As an organiser, I want one clear workspace for courses, tutors and allocation checks so that management work is easy to follow.

Acceptance checks

- The Organiser workspace loads only for the ORGANISER role and keeps loading/empty/error states usable.
- Candidate mark, clash and hour results are visible together before selection.
- The workspace remains keyboard reachable and usable on common phone widths.

Member tasks

- [] Maintain the Organiser navigation and allocation-board shell.
- [] Keep candidate data and rule-result components aligned with the API contract.
- [] Preserve clear loading, empty, error and retry states.

Handoff: Give Member 5 a list of required fields and give Member 4 any reusable table/card needs.

S1-M2-2 | Course management and registered-tutor management UI

Branch: s1/m2/course-tutor-ui | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: S1-M2-1 and S1-M5-2 contract

Goal: As an organiser, I want to manage courses and already-registered tutors using normal account information instead of technical Auth0 identifiers.

Acceptance checks

- Signed-up Tutor accounts appear from the database automatically and can be searched by name or email.
- The Organiser never has to enter, copy or see an Auth0 subject/User ID in the normal Tutor Management UI.
- Edit uses the internal tutor database record to maintain details, weekly-hour limit, marks and busy times.

Member tasks

- [] Maintain course add/edit/remove UI with validation and confirmations.
- [] Maintain tutor search/list/edit UI for self-onboarded Tutor profiles.
- [] Keep the interface professional: no implementation-level Auth0/Neon instructions in normal organiser screens.

Handoff: Member 6 uses the final field list in migrations; Member 5 matches the request/response shapes.

S1-M2-3 | Immediate allocation-rule messages and organiser UI tests

Branch: s1/m2/allocation-rule-ui | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: S1-M2-1 and rule outputs from S1-M5-3

Goal: As an organiser, I want mark, timetable and remaining-hour checks explained before saving so that invalid choices are obvious.

Acceptance checks

- Mark threshold, clash and remaining-hours checks each show written Pass/Fail or controlled unknown wording.
- Save remains unavailable until the selected tutor passes the required checks.
- Colour is never the only way a rule result is communicated.

Member tasks

- [] Maintain the reusable rule-result component.
- [] Test pass/fail/unknown and boundary values.
- [] Keep candidate refresh and selected-tutor state stable after errors.

Handoff: Member 5 supplies stable reason codes; Member 2 maps those codes to plain language without duplicating business logic.

Sprint 1 - Member 3

Tutor dashboard, availability, timesheets, excuses, imports and swaps

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M3-1 | Tutor dashboard using the signed-in tutor identity

Branch: s1/m3/tutor-dashboard | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: Role routing from S1-M1-2 and shared states from S1-M4-2

Goal: As a tutor, I want my own assignments and weekly-hour summary without entering a tutor ID so that another tutor cannot be selected by changing a URL.

Acceptance checks

- GET /api/tutor/dashboard derives the tutor from the verified token subject and returns only that tutor's data.
- A newly onboarded Tutor with no hour limit or allocations gets a usable no-work state rather than an error.
- Loading, empty, error and retry states remain clear.

Member tasks

- [] Maintain the protected tutor dashboard route and cards/table.
- [] Display unset weekly limits safely and never request another tutor by browser-supplied ID.
- [] Test own-data, no-work and failed-load cases.

Handoff: Give Member 5 the dashboard fields and Member 6 the query/seed-data needs.

S1-M3-2 | Tutor availability, work-log and excuse forms on real API routes

Branch: s1/m3/tutor-forms | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: S1-M3-1 and initial API contracts

Goal: As a tutor, I want to record busy times, work logs and excuses through my own authenticated routes so that my records stay accurate.

Acceptance checks

- Tutor workflow endpoints derive ownership from the token and ignore browser-supplied tutorId values.
- Invalid or impossible time/hour values are rejected with clear field messages.
- Valid actions persist through the handwritten Express API and Prisma.

Member tasks

- [] Maintain availability/time-slot form and validation.
- [] Maintain work-log and excuse submission forms.
- [] Test valid, invalid, duplicate/state and cross-user protection cases.

Handoff: Member 5 receives the payload shapes; Member 6 receives required tables and constraints.

S1-M3-3 | Tutor UI contract, responsiveness and accessibility tests

Branch: s1/m3/tutor-ui-tests | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: S1-M3-1, S1-M3-2 and S1-M4-2

Goal: As the team, we want regression tests around the tutor journey so that later changes do not break the working dashboard and forms.

Acceptance checks

- Every required tutor control remains keyboard reachable.
- Required actions are not clipped at the agreed narrow/mobile width.
- Contract or validation changes that break the current tutor journey fail tests.

Member tasks

- [] Keep dashboard/form component tests current.
- [] Cover labels, focus order and responsive layout.
- [] Run the shared test/coverage gate before handover.

Handoff: Give the Project Coordinator the test command/output and give Member 5 any contract failures as bugs.

Sprint 1 - Member 4

Student overflow plus shared responsive and accessible interface

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M4-1 | Student overflow protected UI shell (Sprint 1 mock boundary)

Branch: s1/m4/student-overflow | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: Role routing from S1-M1-2

Goal: As a student, I want a clear protected overflow list so that I can understand unclaimed work while the real Student API is completed later.

Acceptance checks

- Only STUDENT users can open the page.
- Open, closed, empty and failed-load mock states are clear.
- The browser uses the agreed mock/API boundary and never talks directly to Neon.

Member tasks

- ☐ Maintain the Student overflow route/list/cards.
- ☐ Preserve open/closed/empty/error states and phone layout.
- ☐ Keep the mock contract stable until Sprint 2 connects the real backend endpoint.

Handoff: Give Member 5 and Member 6 the exact work-item fields needed by the UI.

S1-M4-2 | Shared role navigation and status components

Branch: s1/m4/shared-navigation | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: Role map from S1-M1-2

Goal: As any Tutor Mx user, I want consistent role navigation and status messages so that the app feels like one product.

Acceptance checks

- Only navigation permitted for the signed-in role is shown.
- Loading, empty, error, success and confirmation states use consistent accessible components.
- Sign out is reachable from Organiser, Tutor and Student workspaces.

Member tasks

- ☐ Maintain shared navigation/status components.
- ☐ Keep visible focus and skip-to-content behavior.
- ☐ Test role visibility and responsive navigation.

Handoff: Members 2 and 3 consume these components; Member 1 supplies the authoritative role list.

S1-M4-3 | Student volunteer confirmation and shared UI tests (Sprint 1 mock boundary)

Branch: s1/m4/volunteer-confirmation | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: S1-M4-1 and S1-M4-2

Goal: As a student, I want a clear confirm/cancel volunteer interaction so that duplicate or accidental claims are prevented.

Acceptance checks

- Confirmation repeats the selected work item and Cancel sends no request.
- Pending state blocks duplicate submission and success/failure is explained.
- Sprint 1 behavior remains on the agreed mock boundary until Sprint 2 real endpoints are ready.

Member tasks

- ☐ Maintain confirm/cancel and pending states.
- ☐ Test duplicate, already-claimed and failure paths.
- ☐ Hand the real request/error expectations to Sprint 2 backend work.

Handoff: Member 5 receives expected duplicate/error responses; the Project Coordinator receives test evidence.

Sprint 1 - Member 5

Handwritten Express API, validation, business rules, security and API tests

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M5-1 | Separate handwritten Express backend and health endpoint

Branch: s1/m5/backend-foundation | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: Chosen backend runtime and repository structure

Goal: As the team, we want a separately runnable handwritten API so that browser code never depends on direct database access.

Acceptance checks

- Backend starts independently and GET /health returns a documented safe response.
- Unhandled routes/errors use the shared safe error shape.
- Environment validation/logging never exposes secrets.

Member tasks

- ☐ Maintain the Express server, central error handling and health/readiness checks.
- ☐ Keep backend lint/test/build scripts working from the repository root.
- ☐ Cover health and error behavior with tests.

Handoff: Give Member 6 the environment and database-connection requirements; give all frontend Members the base URL.

S1-M5-2 | Auth middleware and core profile/course/tutor/tutor-workflow routes

Branch: s1/m5/core-api-contracts | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: S1-M5-1, auth contract from Member 1 and schema from

Goal: As the application, we want protected handwritten endpoints so that all role data reaches React through one authorised API.

Acceptance checks

- Bearer verification and role/ownership middleware protect core routes.
- Course/tutor organiser routes and own-tutor dashboard/forms use Prisma on the backend only.
- Tutor workflow routes derive identity from the token, not a browser tutor ID.

Member tasks

- ☐ Maintain the current-profile, course, tutor and tutor-workflow routes.
- ☐ Keep success/error JSON and status codes consistent.
- ☐ Add success, validation, 401, 403, 404 and conflict tests when contracts change.

Handoff: Members 1-4 use the published route/payload examples; Member 6 aligns migrations and seed data.

S1-M5-3 | Three allocation-rule functions and boundary tests

Branch: s1/m5/allocation-rules | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: Threshold decisions from client/coordinator and Member 6 data

Goal: As an organiser, I want server-side mark, clash and weekly-hour rules so that invalid allocations cannot be bypassed by the UI.

Acceptance checks

- Mark threshold and exact weekly limit follow the agreed inclusive boundaries.
- Touching time ranges are not a clash; true overlaps are.
- Missing/malformed data returns controlled unknown/failure rather than silent approval.

Member tasks

- ☐ Maintain pure rule functions and stable reason codes.
- ☐ Keep boundary tests for threshold, overlap and exact/exceeded hours.
- ☐ Ensure UI consumes server results instead of duplicating business logic.

Handoff: Member 2 maps reason codes to UI text; Member 6 provides candidate/availability/hour data.

Sprint 1 - Member 6

Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

SPRINT 1 BASELINE NOTE

This Member area is already part of the integrated Sprint 1 baseline. Use it to verify/repair the current behavior; do not recreate a second implementation of the same foundation.

S1-M6-1 | Neon PostgreSQL schema, Prisma migrations and safe seed data

Branch: s | Reviewer: assigned reviewer | 4 points | Integrates to: Gitea main

Depends on: Field lists from Members 1-5

Goal: As the team, we want repeatable migrations and safe demo data so that every environment has the same data rules.

Acceptance checks

- Migrations create the required role/profile/course/mark/availability/hour/allocation structures from a clean database.
- Constraints reject invalid relationships and values.
- Seed data is invented/non-personal and no DATABASE_URL is committed.

Member tasks

- [] Maintain versioned Prisma migrations and seed data.
- [] Run schema/database verification after changes.
- [] Coordinate new fields with API and UI owners before migration changes land.

Handoff: Member 5 consumes the backend database connection; Members 2-4 receive stable demo identifiers and fields.

S1-M6-2 | Gitea integration checks, public deployment and shared Sprint 1 URL

Branch: s1/m6/ci-codecov | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: Test scripts from Members 1-5 and repository admin access

Goal: As the team, we want one tested Gitea main plus a public demo so that everyone works from the same source and can demonstrate the integrated system.

Acceptance checks

- Gitea main is the official shared source; members work on branches and the Integration Lead integrates approved work.
- The public frontend loads at tutor-mx.pages.dev and calls the hosted Express API rather than Neon directly.
- Local tests remain required; pushed branches use the shared Gitea Actions runners, and queued jobs are allowed to auto-process.

Member tasks

- [] Maintain Gitea workflow files and honest CI/runner evidence.
- [] Sync approved Gitea main to the GitHub deployment mirror when releasing.
- [] Build the frontend and manually upload the fresh frontend/dist folder to Cloudflare Production when required.

Handoff: All Members use the same test script names; the Integration Lead uses these checks as the merge gate.

S1-M6-3 | South African public-holiday API spike and fallback contract

Branch: s1/m6/external-api-spike | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: Client/coordinator chooses a relevant service and confirms

Goal: As Tutor Mx, we want a safe external public-holiday source to support scheduling without making the core system depend on that service.

Acceptance checks

- The backend-only spike returns a safe subset of South African holiday data.
- Timeout/unavailable/invalid responses return a controlled fallback.
- No user identity, token or database secret is sent to the external service.

Member tasks

- [] Maintain the Nager.Date holiday adapter/spike contract.
- [] Keep mocked success/timeout/invalid-response tests deterministic.
- [] Carry production route/display integration into later sprints rather than redoing the spike.

Handoff: Member 5 will integrate the adapter into the handwritten API; Member 4 will later display the safe response/fallback.

Sprint 1: integration and review gate

- [] Every Member starts from the latest Gitea main and hands over only their own tested branch.
- [] Assigned reviewer reproduces the issue-specific checks and checks page 7/8 evidence.
- [] Integration Lead integrates approved branches into main one at a time and reruns the affected regression/full gate.
- [] After a branch/main push, confirm Gitea Actions completes on a shared runner. A queued job is normal; wait for it. If infrastructure genuinely remains unavailable, record the blocker plus local evidence and do not mark CI green.
- [] After integration, run the relevant three-role/public-environment smoke test.
- [] Update the GitHub deployment mirror and manual Cloudflare Production artifact only from approved Gitea main when a release is required.
- [] Issue is Done only when integrated behavior passes; code merely existing on a Member branch is not Done.

SPRINT 1 SPECIFIC RELEASE CHECK

Verify new Student/Tutor/Organiser account creation -> Auth0 -> automatic onboarding -> correct role dashboard. Verify Organiser Tutor Management contains no Auth0-subject input.

Sprint 2: plan, order and handovers

26 August-15 September 2026 | Complete Basic journeys without rebuilding Sprint 1 foundations

DO NOT REPEAT EARLIER SPRINTS

Sprint 2 starts from the latest approved Gitea main. Do not recreate Auth0 onboarding, core role routing, existing schema/API/UI foundations or manual tutor-ID workflows. Add only the new scope and hardening listed below.

- Finish the remaining Basic end-to-end gaps: Student real API, allocation writes, approvals/statuses and hardening.
- Reuse the working automatic account linking and registered-tutor management from Sprint 1.
- Run representative-user and three-role regression before release.

Member	Sprint ownership
Member 1	Authentication, role security, notifications/search and shared release checks
Member 2	Organiser interface, allocation board, approvals, reports and scenarios
Member 3	Tutor dashboard, availability, timesheets, excuses, imports and swaps
Member 4	Student overflow plus shared responsive and accessible interface
Member 5	Handwritten Express API, validation, business rules, security and API tests
Member 6	Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment
EVERY MEMBER Before handover: use page 7 for automated tests, page 8 for local browser verification, page 9 for the lecturer-runner rule, and page 12 for the Global Definition of Done.	

Sprint 2 - Member 1

Authentication, role security, notifications/search and shared release checks

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M1-1 | Auth lifecycle hardening and email-verification recovery

Branch: s2/m1/account-lifecycle | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: Sprint 1 auth foundation and deployed backend

Goal: As a Tutor Mx user, I want edge cases around verification, reset, sign-out and deletion handled reliably without repeating the working Sprint 1 onboarding flow.

Acceptance checks

- A linked existing account never returns to role choice after normal sign-in.
- An unverified email gets a clear retry/recovery path; add resend-verification support if the agreed Auth0 approach allows it.
- Reset/deletion/network failure cannot leave the app in a false signed-in or half-deleted state.

Member tasks

- [] Keep Sprint 1 self-service onboarding unchanged unless a defect is proven.
- [] Harden verification/retry, reset, deletion and post-action session cleanup.
- [] Add integration tests for failure/recovery paths.

Handoff: Members 2-4 receive a stable signed-in session/role API; Coordinator receives acceptance evidence.

S2-M1-2 | Role and ownership protection across the completed Basic API

Branch: s2/m1/role-protection | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: S2-M1-1 and Member 5 Basic endpoints

Goal: As the school, we want every real Basic endpoint checked for role and ownership so that hidden navigation can never be bypassed.

Acceptance checks

- Organiser-only writes return 403 for Tutor/Student callers.
- Own-data routes do not reveal another Tutor/Student when identifiers are manipulated.
- Frontend route guards and backend authorisation remain consistent.

Member tasks

- [] Create an endpoint-to-role/ownership matrix for the now-real Basic routes.
- [] Add 401/403/cross-user tests for new Sprint 2 endpoints.
- [] Fix permission defects without moving security logic into React.

Handoff: Member 5 receives failing endpoint cases; Members 2-4 use the shared permission/error behavior.

S2-M1-3 | Cross-role security smoke suite and assigned fixes

Branch: s2/m1/security-smoke | Reviewer: Member 3 | 4 points | Integrates to: Gitea main

Depends on: All Sprint 2 Basic journeys integrated on Gitea main

Goal: As the Integration Lead, we want a repeatable three-role smoke suite so that integration cannot break access or expose another role's data.

Acceptance checks

- Separate Student/Tutor/Organiser test accounts land only in their expected workspace.
- Critical endpoints show consistent 401/403 behavior for missing, expired and wrong-role tokens.
- Any severity-one/two security defect blocks release until fixed or formally recorded as a blocker.

Member tasks

- [] Create/run the cross-role smoke sequence against the shared environment.
- [] Open reproducible bugs for failures and fix Member 1-owned defects.
- [] Attach local test evidence plus the completed Gitea Actions run link/status; if queued, wait for the shared runner.

Handoff: Integration Lead runs this before the Integration Lead release decision; Coordinator links results into testing evidence.

Sprint 2 - Member 2

Organiser interface, allocation board, approvals, reports and scenarios

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M2-1 | Organiser course/tutor management hardening - registered tutors only

Branch: s2/m2/organiser-crud | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: Member 5 organiser endpoints and Member 6 Basic schema

Goal: As an organiser, I want the working Sprint 1 management screens hardened around real registered Tutor accounts instead of rebuilding tutor creation.

Acceptance checks

- Tutor search/list comes from self-onboarded Tutor profiles and never asks for Auth0 subject/User ID.
- Edit/validation/refresh behavior stays correct after API failures.
- Course CRUD and tutor details/hour-limit/marks/busy-time edits persist and refresh cleanly.

Member tasks

- ☐ Do not rebuild Sprint 1 Tutor Management or add a manual tutor-create form.
- ☐ Fix validation, empty/error states, responsive layout and stale-data issues found in shared testing.
- ☐ Add integration tests around real course/tutor edit flows.

Handoff: Member 5 fixes contract mismatches; Member 6 validates data constraints.

S2-M2-2 | Live allocation create, edit and remove with server checks

Branch: s2/m2/allocation-board-basic | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: S2-M2-1, Member 5 allocation endpoints and Member 6 queries

Goal: As an organiser, I want the Sprint 1 allocation placeholder replaced by real save/edit/remove behavior while keeping all rule checks visible.

Acceptance checks

- A valid candidate can be saved through a real allocation-write endpoint.
- Failed mark/clash/hour rules are rejected server-side and remain visible in the UI.
- Editing/removing an allocation updates hours and assignments without manual database edits.

Member tasks

- ☐ Connect real allocation create/edit/remove endpoints.
- ☐ Refresh candidate hours and assignment data after every successful change.
- ☐ Test each failed rule plus valid create/edit/remove.

Handoff: Member 5 enforces rules on server; Member 3 sees resulting tutor obligations.

S2-M2-3 | Organiser timesheet and overflow approvals with tests

Branch: s2/m2/organiser-approvals | Reviewer: Member 5 | 4 points | Integrates to: Gitea main

Depends on: Member 3 timesheets, Member 4 volunteer requests and

Goal: As an organiser, I want to approve or return tutor timesheets and approve or reject student volunteer claims so that Basic workflows can complete end to end.

Acceptance checks

- Pending timesheets and claims show the information needed for a decision.
- Approve/reject/return actions persist a clear status and update the responsible user view.
- Duplicate or stale decisions are rejected safely.

Member tasks

- ☐ Build approval queues/views on the Organiser workspace.
- ☐ Connect to the Sprint 2 workflow endpoints and confirmation states.
- ☐ Test valid, rejected, stale and API-failure cases.

Handoff: Members 3 and 4 consume the resulting statuses; Coordinator receives demo-ready accounts/data.

Sprint 2 - Member 3

Tutor dashboard, availability, timesheets, excuses, imports and swaps

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M3-1 | Tutor dashboard hardening and live assignment refresh

Branch: s2/m3/tutor-dashboard-api | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: Member 5 own-dashboard endpoint and Member 2 allocations

Goal: As a tutor, I want the already-live Sprint 1 dashboard to stay accurate when allocations change and when my weekly limit is unset or updated.

Acceptance checks

- The dashboard shows only the signed-in tutor and refreshes after allocation changes.
- No-work and no-hour-limit states remain usable.
- Failed refresh preserves the last safe state and offers retry.

Member tasks

- [] Do not replace the working own-tutor API with a new tutor-ID flow.
- [] Add refresh/status handling for allocation changes.
- [] Expand dashboard tests for changed assignments, unset limits and retry.

Handoff: Member 2 confirms allocation changes; Member 5 fixes data/permission contract issues.

S2-M3-2 | Complete timesheet submission workflow from existing work logs

Branch: s2/m3/timesheet-basic | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: S2-M3-1 and Member 5 work-log/timesheet endpoints

Goal: As a tutor, I want my existing work logs to become a submit/review timesheet workflow so that organisers can approve actual worked hours.

Acceptance checks

- Draft work logs can be grouped/submitted into the agreed timesheet state.
- Invalid hours, duplicate submissions and wrong-session ownership are rejected.
- Tutor sees pending/approved/returned status after organiser action.

Member tasks

- [] Build submit/review-status UI on top of Sprint 1 work-log creation.
- [] Connect real timesheet submit/status endpoints.
- [] Test draft, submit, duplicate, returned and approved cases.

Handoff: Member 2 consumes submitted timesheets; Member 6 validates totals/status constraints.

S2-M3-3 | Excuse status workflow and Tutor Basic regression

Branch: s2/m3/excuse-regression | Reviewer: Member 1 | 4 points | Integrates to: Gitea main

Depends on: S2-M3-1, S2-M3-2 and Member 5 excuse endpoint

Goal: As a tutor, I want the Sprint 1 excuse submission to show its later decision/status and remain reliable with the rest of the Basic journey.

Acceptance checks

- Submitted excuses show pending and later organiser outcome.
- Past/unowned sessions cannot be excused.
- Tutor regression covers dashboard, availability, work log, timesheet and excuse paths together.

Member tasks

- [] Add excuse status/history display rather than recreating submission.
- [] Connect organiser decision outcome once the endpoint exists.
- [] Run the full tutor Basic regression and fix reproducible defects.

Handoff: Coordinator receives a complete tutor demo script result; Member 2 sees excuse/availability impact.

Sprint 2 - Member 4

Student overflow plus shared responsive and accessible interface

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M4-1 | Connect Student overflow to real open-work API

Branch: s2/m4/overflow-api | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: Member 5 overflow endpoint and Member 6 schema

Goal: As a student, I want the Sprint 1 overflow shell to load real open work so that the mock boundary is removed without changing the established UI behavior.

Acceptance checks

- Only eligible/open work is returned for the signed-in Student.
- Closed/claimed work cannot be volunteered for.
- Loading, empty, error and retry states remain clear.

Member tasks

- [] Replace the Sprint 1 mock with the real student overflow endpoint.
- [] Keep the existing responsive list/cards and state components.
- [] Add integration tests for open, closed, empty and failure cases.

Handoff: Member 5 fixes response/permission issues; Member 6 supplies reproducible open/closed seed cases.

S2-M4-2 | Volunteer request and outcome statuses

Branch: s2/m4/volunteer-basic | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: S2-M4-1 and Member 5 volunteer endpoint

Goal: As a student, I want my confirmed volunteer request stored and its outcome shown so that I know whether I received the work.

Acceptance checks

- Confirm creates one request and duplicate clicks/claims are controlled.
- Pending/approved/rejected/already-claimed statuses are visible.
- Another Student cannot view or mutate the request.

Member tasks

- [] Connect the Sprint 1 confirmation UI to the real volunteer endpoint.
- [] Show persisted request/outcome states.
- [] Test duplicate, race, rejected and permission cases.

Handoff: Member 2 consumes pending claims; Member 5/6 enforce uniqueness and atomic outcome.

S2-M4-3 | External-result UI, accessibility and Student Basic regression

Branch: s2/m4/shared-quality | Reviewer: Member 6 | 4 points | Integrates to: Gitea main

Depends on: Member 6 external adapter and all Basic shared UI

Goal: As a user, I want the approved external holiday result/fallback and Student journey presented accessibly without weakening the working shared UI.

Acceptance checks

- External information is clearly supporting information and a safe fallback appears when unavailable.
- Student primary actions remain keyboard/mobile accessible.
- Student regression covers real overflow and volunteer flows.

Member tasks

- [] Render only the safe external API response/fallback from the backend.
- [] Run accessibility/responsive checks on Student and shared components.
- [] Fix Member 4-owned Basic regressions found in representative-user testing.

Handoff: Coordinator receives accessibility/test evidence; Member 6 receives any adapter failure cases.

Sprint 2 - Member 5

Handwritten Express API, validation, business rules, security and API tests

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M5-1 | Allocation-write and organiser-approval API completion

Branch: s2/m5/organiser-api | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: Member 6 Basic schema and Member 1 role matrix

Goal: As the application, we want real allocation mutations and organiser approval routes so Sprint 2 can complete workflows without redoing Sprint 1 course/tutor endpoints.

Acceptance checks

- Allocation create/edit/remove rechecks mark, clash and hours on the server inside safe database operations.
- Timesheet/claim organiser decisions enforce allowed state transitions.
- Legacy POST /api/tutors is not required by the normal self-service Tutor UI.

Member tasks

- [] Implement allocation mutation routes and transaction/service logic.
- [] Implement organiser timesheet/claim decision routes.
- [] Add validation, permission, conflict and state-transition tests.

Handoff: Member 2 connects the board; Member 6 tunes queries/constraints.

S2-M5-2 | Student overflow/volunteer and timesheet-status API

Branch: s2/m5/tutor-student-api | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: Member 6 Basic workflow schema and Member 1 ownership rules

Goal: As the frontend, we want the missing Student and workflow endpoints so the remaining Sprint 1 mocks/status gaps can become real.

Acceptance checks

- Student open-work and volunteer routes are role/ownership safe.
- Timesheet submit/status and excuse decision/status contracts are stable.
- Duplicate/race/state conflicts use safe 409-style responses.

Member tasks

- [] Implement Student overflow/volunteer endpoints.
- [] Complete timesheet/excuse status routes needed by Tutor/Organiser UI.
- [] Add success, validation, 401/403/404/409 tests.

Handoff: Members 3 and 4 connect their screens; Member 6 enforces database uniqueness/transitions.

S2-M5-3 | API contract, deployment and coverage hardening

Branch: s2/m5/api-quality | Reviewer: Member 2 | 4 points | Integrates to: Gitea main

Depends on: S2-M5-1, S2-M5-2 and CI from Member 6

Goal: As the team, we want the expanded Sprint 2 API to keep the existing safe error contract and remain deployable/testable as one service.

Acceptance checks

- New routes use the established { error: { code, message } } shape and do not leak stack traces/secrets.
- Hosted health/smoke behavior stays stable after deployment.
- Coverage includes critical success, permission, validation and conflict branches.

Member tasks

- [] Standardise only new/changed routes; do not rewrite already-stable Sprint 1 contracts unnecessarily.
- [] Run backend build/smoke/coverage and fix real regressions.
- [] Provide deployment and API evidence to the Integration Lead.

Handoff: Integration Lead uses health/tests/Codecov as merge evidence; Coordinator uses route examples for API docs.

Sprint 2 - Member 6

Neon PostgreSQL, Prisma migrations, external integration, data engine, CI/deployment

REQUIRED BEFORE HANDOVER

See page 7 (run tests), page 8 (check the feature locally), page 9 (Actions runner rule) and page 12 (Global Definition of Done).

CARRY FORWARD

Start from the latest Gitea main and reuse the working Sprint 1 implementation. The three cards below are the new work for this sprint.

S2-M6-1 | Sprint 2 migrations for workflow state and allocation writes

Branch: s2/m6/basic-workflow-schema | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: Final Basic payloads from Members 2-5

Goal: As the team, we want only the new Sprint 2 data/state additions migrated so the working Sprint 1 schema is extended rather than recreated.

Acceptance checks

- New allocation mutation, timesheet/approval, excuse-status and volunteer-request data is represented with safe constraints.
- Migrations apply from the existing Sprint 1 schema and from a clean database.
- No manual production table edits are required.

Member tasks

- ☐ Add minimal versioned migrations for new Sprint 2 states/relations.
- ☐ Update seed data for approval and race/conflict cases.
- ☐ Run db verification and coordinate schema changes with API owners.

Handoff: Member 5 connects transactions/queries; Members 2-4 use stable demo data.

S2-M6-2 | Production public-holiday adapter with graceful fallback

Branch: s2/m6/external-api-adapter | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: Approved Sprint 1 external service and Member 5 backend

Goal: As Tutor Mx, we want the Sprint 1 holiday spike exposed safely through the backend with caching/fallback so the UI can use it without repeating the research spike.

Acceptance checks

- Only the backend calls the external service and returns the approved safe fields.
- Timeout/unavailable/invalid-response uses the established fallback.
- The integration does not block core Tutor Mx workflows.

Member tasks

- ☐ Turn the validated Sprint 1 spike into a production backend adapter/route.
- ☐ Add sensible caching and deterministic mocked tests.
- ☐ Document the fallback and any service limits.

Handoff: Member 5 exposes the adapter through handwritten API; Member 4 renders success/fallback.

S2-M6-3 | Allocation data performance, deployment smoke and Codecov reliability

Branch: s2/m6/data-delivery-quality | Reviewer: Member 4 | 4 points | Integrates to: Gitea main

Depends on: S2-M6-1, Member 5 API and Gitea Actions

Goal: As the Integration Lead, we want the completed Basic data flow and release checks reliable enough for repeated integration.

Acceptance checks

- Allocation candidates/writes and dashboard refresh use efficient queries with correct results.
- Clean setup/build/smoke works against the shared environment.
- Gitea Actions/coverage runs on the shared lecturer runners; queued jobs are expected and completed run evidence is retained.

Member tasks

- ☐ Profile/fix obvious allocation query issues and database constraints.
- ☐ Run clean migration/seed/check/smoke before release.
- ☐ Maintain Gitea CI/coverage config, verify the shared-runner result, and never invent green CI evidence.

Handoff: Integration Lead uses this as Sprint 2 merge gate; Coordinator receives run links and screenshots.

Sprint 2: integration and review gate

- [] Every Member starts from the latest Gitea main and hands over only their own tested branch.
- [] Assigned reviewer reproduces the issue-specific checks and checks page 7/8 evidence.
- [] Integration Lead integrates approved branches into main one at a time and reruns the affected regression/full gate.
- [] After a branch/main push, confirm Gitea Actions completes on a shared runner. A queued job is normal; wait for it. If infrastructure genuinely remains unavailable, record the blocker plus local evidence and do not mark CI green.
- [] After integration, run the relevant three-role/public-environment smoke test.
- [] Update the GitHub deployment mirror and manual Cloudflare Production artifact only from approved Gitea main when a release is required.
- [] Issue is Done only when integrated behavior passes; code merely existing on a Member branch is not Done.

SPRINT 2 SPECIFIC RELEASE CHECK

Verify real Student overflow/volunteer, allocation create/edit/remove and organiser approval/status journeys end to end.
Confirm no Sprint 1 manual-ID flow was reintroduced.

Post-Sprint 2 production stabilisation

15-16 September 2026 | Live feedback fixes that become the Sprint 3 baseline

HISTORICAL RECORD RULE

Sprint 1 and Sprint 2 issue cards remain the record of what was planned and delivered in those sprints. The items below were discovered through integrated/live testing after the Sprint 2 review window and are recorded as the baseline Sprint 3 inherits.

Why this work was needed

- The merged product worked end to end, but some cross-layer behaviour was confusing or inconsistent when tested as a real Tutor or Organiser.
- The team chose narrow fix branches with regression tests rather than reopening completed Sprint 1/2 foundations.
- The main themes were ownership (who sets a Tutor schedule/capacity), visibility (calendar/workload presentation), candidate API correctness and post-save state consistency.

Chronological changes

Date	Change	Result / new baseline
15 Sep	Final Sprint 2 stability pass	A failing organiser integration was reverted rather than shipped; passing cache/timesheet corrections were retained. Course deletion, organiser layout and timetable/candidate behaviour were retested.
15 Sep	Tutor-owned work preferences	Tutor sets weekly work capacity and Busy/Class/Study/Other unavailable periods. Organiser views this read-only.
15 Sep	Weekly calendar + detail resilience	Tutor and Organiser share a weekly schedule calendar. Marks and schedule load independently. Optional legacy time-slot fields no longer hide valid marks. Confusing UTC-week wording was removed.
16 Sep	Candidate API contract repair	The query already returned clashDetails, but server mapping omitted it. The mapping now includes clashDetails so the frontend validator accepts valid candidate responses.
16 Sep	Post-save self-conflict repair	The saved allocation ID is passed as excludeAllocationId during refresh so a newly created/edited confirmed allocation does not clash with itself. Genuine conflicts still block.
16 Sep	Empty-state clarification	Only tutors passing mark + availability + capacity are selectable. If none pass, the board states "No tutors currently available"; failed tutors remain read-only with reasons.
16 Sep	Direct week calendar navigation	Organiser workload keeps Previous / This week / Next controls and adds Go to week. Any chosen date resolves to its Monday-Sunday week and reloads the same server-supplied workload data.
16 Sep	Completed-session work-log gate	Tutor work logs are available only after the confirmed session occurrence has ended. Future/in-progress occurrences are disabled in the UI and the backend rejects early submissions; recurring sessions are checked per occurrence.

Sprint 3 carry-forward

- Preserve the completed Previous / This week / Next controls and Go to week date picker while adding ranking, shortages and reports.
- Build timetable import and recurring exceptions into the existing Tutor-owned calendar instead of creating a second schedule model.
- Preserve the completed-session work-log guard while extending timesheet corrections, disputes and payroll/export workflow.

Sprint 3-4 current-main code audit

18 September 2026 | Audited against supplied main snapshot 02e8bc3d1304275550cb7abb89ebc7dee1024b6f

WHY THIS ROADMAP WAS REWRITTEN

The previous Sprint 3-4 plan still contained work that current main already does, and several early UI cards depended on APIs/database work assigned to Members 5-6 later. This edition removes that duplication and makes every Member turn independently completable before the next Member starts.

CURRENT MAIN - KEEP THESE AS FOUNDATIONS

Area	What the code already proves
Accounts/security	Auth0 sign-up/sign-in, one saved role, protected routes, reset/delete and automatic Profile linking already exist.
Tutor schedule/capacity	Tutor-owned weekly capacity, Busy schedule CRUD, WEEKLY/ONCE recurrence, repeat-until dates and shared weekly calendar already exist.
Organiser workload	Tutor search by name/email, weekly capacity/allocated/remaining/worked cards and Previous/This/Next/Go to week navigation already exist.
Allocation	Single board, mark + clash + selected-week capacity checks, clashDetails, allocation create/edit/remove and self-conflict exclusion already exist.
Tutor workflow	Completed-session work-log guard, timesheet submit/return/resubmit/status, excuse submit/status and organiser excuse decision backend already exist.
Student workflow	Open work, volunteer request, persisted outcomes and backend withdrawal endpoint already exist.
External/release	Public-holiday fallback, shared CI/check commands and deployed frontend/backend flow already exist.

DUPLICATES REMOVED OR NARROWED

- Ranking is no longer "start ranking from zero": current SQL already orders eligible candidates by mark, so Sprint 3 upgrades this into explicit multi-factor, explainable ranking.
- The generic capacity/allocated/worked report is removed: those figures are already on Tutor Management. Sprint 3 reporting now targets missing brief items - hours per course, budget/spend, workload distribution and shortages.
- Student withdrawal is no longer a new backend task: the PATCH route already exists. Sprint 3 only wires it into the Student UI and hardens the state/conflict experience.
- Timetable work extends the existing TimeSlot recurrence/calendar instead of creating another schedule model.
- Member 5/6 no longer build prerequisite APIs/migrations after the UI members. Each feature owner includes the small backend/schema changes their feature needs on the same branch.

Roadmap rules and project-brief coverage

Every required Basic, Intermediate and Advanced capability has an owner before final release

STRICT CHRONOLOGICAL METHOD + INTEGRATION LEAD SEPARATION

Member 1 starts from current main and completes only their assigned feature work on one branch. They self-test that branch and hand it to the Integration Lead. The Integration Lead alone reviews the handover, merges it to main, resolves integration conflicts and checks updated main. Only after that merge does Member 2 pull main and start. Repeat Member 1 -> 2 -> 3 -> 4 -> 5 -> 6 in Sprint 3, then the same order in Sprint 4. Members do not merge to main, run the cross-system release gate, sync the deployment mirror or redeploy production; those are Integration Lead responsibilities.

NO-FORWARD-DEPENDENCY + INTEGRATION-LEAD RULE

If a Member feature needs a small route, table, migration, validator or service, that same Member adds it on the same branch so later Members are never prerequisites. Each Member tests only their own changed feature and provides evidence. The Integration Lead is separate from the six Member workloads: the Integration Lead alone reviews every handover, merges to Gitea main, reruns the affected cross-system checks, fixes/coordinates merge conflicts, confirms main is healthy, syncs the GitHub deployment mirror and performs production deployment verification.

PROJECT BRIEF -> REVISED OWNER

Brief area	Where it is completed
Basic gaps	Organiser excuse review UI -> S3 M2. Tutor digital submit declaration -> S3 M3. Organiser overflow posting/closing -> S3 M4.
Intermediate allocation	Explicit ranking + shortages + bulk preview/commit + reasoned automation override -> S3 M2.
Intermediate availability	CSV/ICS/paste timetable import + recurring entries bounded by an academic term -> S3 M3.
Intermediate timesheets	Corrections, disputes, due reminders and payroll-ready approved export -> S3 M1 + M3.
Intermediate visibility	Notifications for assignment/status/due events -> S3 M1. Course/person/timesheet search -> S3 M1.
Intermediate reporting	Hours per course, spend vs budget and workload-distribution data -> S3 M2; Command Centre heatmap/gauges/fairness visualisation -> S3 M6.
Advanced planning	Draft side-by-side scenarios, locks, compare/publish -> S4 M2.
Advanced collaboration	Stale-write conflict handling plus visible organiser presence -> S4 M2.
Advanced swaps	Constraint-checked Tutor swaps and approval -> S4 M3.
Advanced audit	Mutation audit trail + allocation restore as a new auditable change -> S4 M5.
Advanced allocator	Whole-school explainable proposal engine balancing soft objectives while hard rules stay fixed -> S4 M6.

HARD-RULE DECISION

Mark qualification, timetable/confirmed-allocation clashes and selected-week capacity remain non-overridable hard constraints. The organiser may override the automation recommendation/order/selection with a written reason, but cannot force an allocation that fails a hard eligibility rule. This keeps ranking, bulk, scenarios, swaps and proposals consistent.

Sprint 3: audited plan, order and handovers

16-29 September 2026 | *Intermediate features added to the current working system*

SPRINT 3 MAIN IDEA

Do not rebuild current main. Sprint 3 fills the remaining Intermediate brief, improves the organiser/tutor/student experience and creates the data needed for Advanced Sprint 4 - while preserving all working Basic flows.

EXACT ORDER

Member	Only new Sprint 3 work
Member 1	Notifications + due reminders; secure course/person/timesheet search + Ctrl/Cmd+K palette; guided Tutor setup. These become the Command Centre alert/search layer.
Member 2	Staffing requirements, explainable eligible ranking, shortage calculations, bulk allocation, course-hours/budget/workload data and Organiser excuse review. Build the planning logic/data that later Command Centre visuals consume.
Member 3	Timetable CSV/ICS/paste import + term bounds; returned-timesheet corrections/disputes/declaration; approved payroll export. Keep the schedule/timesheet data feeding Command Centre analytics accurate.
Member 4	Wire Student withdrawal; Organiser overflow post/edit/close; Send to overflow from unmet staffing work; polish the new Student/overflow interactions for keyboard and small screens.
Member 5	Harden the new Sprint 3 API contracts, permissions, transactions, retry/race handling, limits and performance. This is API engineering on the Member 5 branch - not main integration/release work.
Member 6	Build the visible Organiser Command Centre: live staffing KPI cards, shortage heatmap, budget/workload/fairness visuals, drill-downs and a Needs Attention action feed with quick links. This is real feature work; Member 6 does not merge, release or deploy.

STAND-OUT SPRINT 3 POLISH ALREADY BUILT INTO THE PLAN

- Allocation Command Centre: Member 6 assembles live headline cards for allocated / required sessions, remaining shortages, spend vs budget and outstanding work using the APIs/data already added earlier in the sprint.
- Shortage heatmap + budget/workload/fairness visuals: Member 6 turns Member 2 report/requirement data into clear drill-down visuals; written numbers and labels always accompany colour.
- Ctrl/Cmd+K command palette: jump quickly to permitted courses, people and timesheets from anywhere, including the Command Centre.
- CSV/paste + optional .ics timetable import with preview/error rows before saving, so Tutor availability is fast to enter and safe to correct.
- One controlled Send to overflow action converts an unmet staffing requirement into Student volunteer work; all new screens remain keyboard-first and mobile-friendly.

STARTING SAFETY CHECK

Before coding any card, ask: "Does current main already do this?" If yes, treat it as regression evidence or a narrow improvement - not a second implementation.

Sprint 3 - Member 1

BEGINNER-FRIENDLY SUMMARY: Your goal is to make Tutor Mx tell each user what needs attention and help them get there quickly. Build the notification inbox, due reminders, secure global search/Ctrl+K palette and Tutor setup checklist so users do not have to hunt through the site or guess what to do next.

STARTING POINT / DO NOT REPEAT

Start from current approved main. Do not rebuild Auth0 onboarding, role choice, account settings, route guards or the existing Tutor capacity screen. Any Notification/search storage or API needed for these cards is included on this same Member 1 branch.

S3-M1-1 | Notification inbox and due reminders

BEGINNER GOAL	Build a secure notification inbox with read/unread state and due reminders so that each user immediately sees assignment/status changes and overdue actions without duplicate alerts.
USER ACCEPTANCE	Given: a signed-in user is affected by an assignment/status event or a reminder becomes due When: the event occurs or reminder job/check runs Then: one correct notification appears; it can be marked read and its link opens only authorised data
ACCEPTANCE TESTS	- One event/reminder creates one recipient-specific notification; retries do not duplicate it. - Assignment, timesheet, excuse and volunteer outcome changes notify only affected users. - Due reminders identify the person currently holding up the workflow without leaking private data.
UNIT TESTS + TASKS	Unit: notification identity/de-dup helper is deterministic Unit: target permission mapper rejects another user/role [] Add the minimal Notification model/migration, routes and inbox/unread badge on this branch. [] Emit notifications from existing workflow state changes without changing their business rules. [] Add reminder generation and read/unread tests, including stale/deleted targets.

S3-M1-2 | Global search + Ctrl/Cmd+K command palette

BEGINNER GOAL	Build one role-filtered search API plus a Ctrl/Cmd+K command palette so that Organisers, Tutors and Students can jump straight to permitted courses, people and timesheets instead of hunting through menus.
USER ACCEPTANCE	Given: a Student, Tutor or Organiser searches by text When: they use the search box or Ctrl/Cmd+K palette Then: only result categories/records permitted for that role appear and selecting a result opens the correct safe destination
ACCEPTANCE TESTS	- Search covers course/person/timesheet where the role is allowed to see them. - Results are bounded/paginated and stale or forbidden results show a safe state. - Keyboard open, navigate, close and focus-return behaviour is usable.
UNIT TESTS + TASKS	Unit: role/category filter removes disallowed results Unit: query normaliser and result grouping handle empty/limit boundaries [] Add search endpoint/query + UI on this branch; reuse existing auth/ownership middleware. [] Add command palette with grouped results, loading/empty/error states. [] Test wrong-role, cross-user, deleted record and keyboard flows.

S3-M1-3 | Guided Tutor setup + shared regression

BEGINNER GOAL	Build a Tutor setup checklist and re-test shared auth/accessibility so that new Tutors know exactly whether capacity or schedule setup is missing and existing security/navigation stays unchanged.
USER ACCEPTANCE	Given: a new Tutor has not completed capacity and/or schedule setup When: they open their dashboard/setup checklist Then: the app shows exactly what remains and deep-links to the existing Work Preferences/calendar; completing an item updates the checklist
ACCEPTANCE TESTS	- Checklist checks existing capacity/schedule state; it never changes the Tutor role/Profile. - Dismiss/complete state does not hide required validation from Organiser allocation rules. - Shared auth 401/403, keyboard focus and labels still pass after Member 1 changes.
UNIT TESTS + TASKS	Unit: setup-state helper identifies missing capacity/schedule correctly Unit: changed shared components keep focus/labels in tests [] Build a compact setup checklist using existing endpoints/screens. [] Run shared route/auth/accessibility regression and fix only reproduced Member 1 defects. [] Keep issue -> fix -> test evidence on s3/m1/completion.

ONE-BRANCH HANDOFF

Keep all Member 1 Sprint 3 work on s3/m1/completion. Self-test only the work on this branch, push it and hand evidence to the Integration Lead. Do not merge to main. The Integration Lead alone reviews/tests the handover, merges it to main and checks updated main. Only then does Member 2 pull that main and create s3/m2/completion.

Sprint 3 - Member 2

BEGINNER-FRIENDLY SUMMARY: Your goal is to build the staffing and allocation logic that the Organiser needs so the system knows what work is still required, which eligible Tutors are stronger choices, how to fill obvious work in bulk, and what the real hours/budget/workload totals are. Keep this feature usable on its own; Member 6 will later turn your live data into the polished Command Centre overview.

STARTING POINT / DO NOT REPEAT

Start from main after Member 1 is merged. Do not rebuild allocation CRUD, week navigation, candidate hard rules, clashDetails, self-conflict exclusion or existing per-Tutor workload cards. Any small schema/API needed by this Organiser feature is included on Member 2 branch.

S3-M2-1 | Staffing requirements + explainable ranking + shortage calculations

BEGINNER GOAL	Build real staffing requirements, shortage calculations and explainable eligible ranking so that Tutor Mx knows exactly what work still needs Tutors and can show why one eligible Tutor ranks above another.
USER ACCEPTANCE	Given: an Organiser opens required tutoring work When: requirement status and candidate ranking load Then: ineligible Tutors stay blocked with reasons, eligible Tutors are ordered consistently, and the system returns the exact remaining shortage for later display anywhere in the UI.
ACCEPTANCE TESTS	- A StaffingRequirement records course, session/period, hours/slots and status; it is demand, not a second Allocation model. - Ranking never changes isEligible=false; soft factors can include mark strength, remaining capacity and current load/fairness. - Shortage is calculated from unmet requirement and is returned as reusable data; a polished heatmap is Member 6 work.
UNIT TESTS + TASKS	Unit: ranking/tie-breaker returns stable order Unit: shortage helper covers zero candidates, partial fill and week boundaries [] Add requirement storage/API and a clear Organiser requirement/status view. [] Upgrade candidate display with factor chips/score explanation. [] Expose reusable shortage data for Member 6; do not build the final heatmap here. [] Keep failed-Tutor reasons and hard rules unchanged.

S3-M2-2 | Bulk allocation preview/commit + reasoned automation override

BEGINNER GOAL	Build bulk allocation preview/commit with recorded manual-choice reasons so that Organisers can fill obvious work quickly without bypassing hard rules or accidentally writing data before confirmation.
USER ACCEPTANCE	Given: several open requirements have eligible candidates When: the Organiser previews, includes/excludes items, optionally chooses a different eligible Tutor with a reason, then confirms Then: preview writes nothing; commit reports each success/conflict and stores any manual-decision reason
ACCEPTANCE TESTS	- Preview has zero database side effects and shows chosen Tutor + reasons. - Commit is transaction-safe per item, rechecks hard rules and supports partial conflict reporting. - Written reason is required when the Organiser deviates from the automatic eligible recommendation.
UNIT TESTS + TASKS	Unit: bulk selection helper never mutates saved allocations Unit: manual-choice validator requires a reason and rejects ineligible target [] Build preview/select/confirm/result UI and supporting route/service on this branch. [] Preserve selected week/context after conflicts. [] Test partial failure, stale requirement, duplicate confirm and reason capture.

S3-M2-3 | Course hours, budget/spend, workload data + excuse queue

BEGINNER GOAL	Build the real course-hours, spend/budget and workload-distribution calculations plus the Organiser excuse queue so that later Command Centre cards/charts can use one trustworthy backend definition instead of calculating totals differently in the browser.
USER ACCEPTANCE	Given: an Organiser chooses a period/course or opens approvals When: they view analytics or review a pending excuse Then: hours per course, approved spend vs budget, Tutor work distribution and shortages are clear; excuse approve/reject updates the Tutor status
ACCEPTANCE TESTS	- Report data separates capacity, allocated hours and approved worked hours and labels the selected period. - Add a minimal course budget + approved-hour cost basis so spend is reproducible and auditable. - API returns the live values Member 6 needs for gauges/charts; Member 2 does not own the polished visual dashboard. - Excuse queue uses the existing decision endpoint and handles stale/duplicate decisions safely.
UNIT TESTS + TASKS	Unit: analytics aggregation keeps allocated/worked/spend separate Unit: zero/over-budget and distribution calculations are deterministic [] Add report endpoint + simple Organiser report view + minimal budget fields. [] Add Organiser excuse queue/actions to the existing approvals workspace. [] Expose stable report payloads for Member 6 visualisation. [] Test filters, totals, over-budget, empty data and excuse approve/reject.

ONE-BRANCH HANDOFF

Keep all Member 2 Sprint 3 work on s3/m2/completion. Self-test only the work on this branch, push it and hand evidence to the Integration Lead. Do not merge to main. The Integration Lead alone reviews/tests the handover, merges it to main and checks updated main. Only then does Member 3 pull that main and create s3/m3/completion.

Sprint 3 - Member 3

BEGINNER-FRIENDLY SUMMARY: Your goal is to make Tutor availability and timesheets easy to enter and trustworthy so the allocation and Command Centre numbers are based on good data. Add timetable import, term limits, returned-timesheet corrections/disputes/declaration and an approved-work export without replacing the existing Tutor calendar or workflow.

STARTING POINT / DO NOT REPEAT

Start from main after Member 2 is merged. Reuse Tutor weekly capacity, existing TimeSlot CRUD/WEEKLY/ONCE recurrence, repeat-until, weekly calendar, completed-session work-log guard and current RETURNED timesheet resubmit flow.

S3-M3-1 | CSV/ICS/paste timetable import into existing schedule + term calendar

BEGINNER GOAL	Build CSV/paste and optional ICS timetable import into the existing TimeSlot calendar with academic-term bounds so that Tutors can provide availability quickly without creating a second schedule system.
USER ACCEPTANCE	Given: a Tutor has timetable rows/file and an active academic term When: they preview parsed rows, fix/skip invalid rows and confirm Then: valid rows become normal TimeSlots visible in the existing calendar; invalid/duplicate rows change nothing; recurrence does not silently run beyond the term
ACCEPTANCE TESTS	- Preview shows local date/time, label, source and recurrence before saving. - Support paste/CSV and, where practical, .ics import without introducing a second timetable model. - Existing manual and imported TimeSlots cause identical allocation clash behaviour.
UNIT TESTS + TASKS	Unit: parser returns row-level errors for bad dates/times/categories Unit: import mapper normalises CSV/ICS into existing TimeSlot payloads and term bounds [] Add minimal AcademicTerm data/API and import preview/save on this branch. [] Use existing TimeSlot routes/service rules; add batch endpoint only if needed for atomic import. [] Test duplicate import, partial invalid rows, local time, recurrence end and rollback.

S3-M3-2 | Returned-timesheet corrections, digital declaration and disputes

BEGINNER GOAL	Build returned-timesheet correction, declaration, revision history and dispute controls so that Tutors can fix rejected submissions transparently without erasing previous values or Organiser reasons.
USER ACCEPTANCE	Given: a Tutor has a RETURNED timesheet with an Organiser note When: they correct permitted work-log values, acknowledge the declaration and resubmit or raise a dispute Then: previous values/reasons remain traceable, illegal locked states stay locked, and the new state is visible to Tutor/Organiser
ACCEPTANCE TESTS	- Only DRAFT/RETURNED work can be edited; SUBMITTED/APPROVED remain locked. - Corrections preserve prior values/reasons in revision history instead of overwriting evidence. - A dispute records author/time/reason/status and the resubmit declaration is stored with the submission.
UNIT TESTS + TASKS	Unit: state/editability helper allows only legal transitions Unit: revision/diff helper preserves old/new hours and description [] Add correction, revision/declaration and dispute storage/API/UI on this branch. [] Show a compact timeline of return reason, corrections, dispute and resubmission. [] Test ownership, stale state, multiple returns, locked edits and declaration required.

S3-M3-3 | Payroll-ready approved export + full Tutor regression

BEGINNER GOAL	Build payroll-ready approved-work export and run the full Tutor regression so that payroll/report totals are trustworthy and the new data feeding Command Centre analytics remains consistent.
USER ACCEPTANCE	Given: a period contains approved and unapproved Tutor work When: the Tutor/authorised Organiser exports the approved period and the full Tutor regression runs Then: export totals match approved records only; unapproved work is excluded; imported/manual schedule and workflow rules still agree
ACCEPTANCE TESTS	- CSV export has clear period, Tutor, course, hours and approval fields. - Approved totals reconcile with the same backend data shown in the app. - Regression covers setup, schedule/import, allocations, completed-session logging, excuses, corrections/disputes and notifications.
UNIT TESTS + TASKS	Unit: approved-work aggregation excludes non-approved states Unit: CSV formatter escapes fields and produces deterministic columns/totals [] Add approved export route/download and tests on this branch. [] Run clean-browser Tutor journey and fix only Member 3 defects. [] Capture evidence for term boundary, returned correction, dispute and export totals.

ONE-BRANCH HANDOFF

Keep all Member 3 Sprint 3 work on s3/m3/completion. Self-test only the work on this branch, push it and hand evidence to the Integration Lead. Do not merge to main. The Integration Lead alone reviews/tests the handover, merges it to main and checks updated main. Only then does Member 4 pull that main and create s3/m4/completion.

Sprint 3 - Member 4

BEGINNER-FRIENDLY SUMMARY: Your goal is to finish the Student overflow experience so a shortage can become real volunteer work and a Student can safely withdraw a pending request. Build the Organiser overflow controls and make the screens you change responsive/accessibile. Test only the flows you own; the Integration Lead will later run the combined-main regression.

STARTING POINT / DO NOT REPEAT

Start from main after Member 3 is merged. Student open-work, volunteer submission/outcomes and the backend withdrawal route already exist. Do not recreate them. Member 4 wires the missing UI and adds the Organiser publishing/management journey.

S3-M4-1 | Wire Student withdrawal UI to the existing backend route

BEGINNER GOAL	Connect the existing withdrawal backend to the Student UI so that a Student can safely withdraw only their own pending volunteer request and immediately see the server's final state.
USER ACCEPTANCE	Given: a Student has their own PENDING volunteer request When: they choose Withdraw and confirm Then: the existing PATCH endpoint runs, the latest server status becomes WITHDRAWN, and non-withdrawable/wrong-owner requests cannot show a working action
ACCEPTANCE TESTS	- Withdraw action appears only for the current Student in a withdrawable state. - Conflict/state response refreshes to the server truth and explains what changed. - Existing Pending/Approved/Rejected cards and duplicate/race protections remain intact.
UNIT TESTS + TASKS	Unit: UI eligibility helper enables withdrawal only for PENDING own request Unit: response mapper displays WITHDRAWN/conflict clearly [] Add withdraw API client method + button/confirm UI only; reuse existing backend route. [] Refresh open work and request lists after success/conflict. [] Test cancel, success, stale, approved/rejected and wrong-owner paths.

S3-M4-2 | Organiser post/edit/close overflow work + requirement-to-overflow action

BEGINNER GOAL	Build Organiser overflow post/edit/close screens plus a Send to overflow action from an unmet staffing requirement so that shortages identified in the Command Centre can become real volunteer work without manual database setup.
USER ACCEPTANCE	Given: an Organiser has unfilled tutoring work or an existing overflow item When: they post/edit/close it, or convert an unmet requirement to overflow Then: Students see only open eligible work; closing/stale edits are safe and the source requirement remains traceable
ACCEPTANCE TESTS	- Organiser-only create/edit/close endpoints and screens validate date/hour/course/state. - One-click "Send to overflow" can prefill from an unmet requirement without changing existing allocations. - Approving a Student claim keeps the existing race/uniqueness rules and updates the visible status.
UNIT TESTS + TASKS	Unit: overflow state transition helper rejects illegal close/edit actions Unit: requirement-to-overflow mapper preserves course/time/hours and source reference [] Add Organiser overflow management API/UI on this branch. [] Add explicit OPEN/CLOSED status, confirmation and stale/version checks. [] Test post, edit, close, source conversion, already-claimed and two-session stale cases.

S3-M4-3 | Accessible responsive polish + representative user testing

BEGINNER GOAL	Polish and user-test the Student/overflow/shared UI pieces changed by Member 4 so that volunteer, withdrawal, overflow posting and related dialogs work by keyboard and on common screen sizes.
USER ACCEPTANCE	Given: representative users follow the Student/overflow flows changed on this branch When: they use keyboard and common phone/desktop widths Then: those Member 4 flows are understandable, primary actions stay visible and high-priority Member 4 issues are fixed/retested.
ACCEPTANCE TESTS	- No horizontal page scrolling or clipped primary actions in Member 4 Student/overflow screens. - Dialogs restore focus and announce success/error/conflict with text. - Member 4 captures focused user evidence; the Integration Lead owns the later whole-system acceptance pass.
UNIT TESTS + TASKS	Unit: shared focus/status helpers cover success/error/conflict Unit: responsive components cover long names/table content [] Test the Student/overflow screens changed on this branch. [] Fix only Member 4/shared UI defects. [] Save before/after screenshots and focused retest evidence for handover.

ONE-BRANCH HANDOFF

Keep all Member 4 Sprint 3 work on s3/m4/completion. Self-test only the work on this branch, push it and hand evidence to the Integration Lead. Do not merge to main. The Integration Lead alone reviews/tests the handover, merges it to main and checks updated main. Only then does Member 5 pull that main and create s3/m5/completion.

Sprint 3 - Member 5

BEGINNER-FRIENDLY SUMMARY: Your goal is to strengthen the new Sprint 3 backend so the feature APIs are safe under wrong roles, invalid input, retries and competing requests. You are not responsible for merging everyone, proving the whole main branch or deploying. Test the API areas you change and hand the branch to the Integration Lead.

STARTING POINT / DO NOT REPEAT

Start from main after Member 4 is merged. Members 1-4 already brought the routes/schema needed for their vertical features. Member 5 must not rewrite those features; this turn standardises, secures and hardens the integrated API.

S3-M5-1 | Integrated Sprint 3 API contract + permission audit

BEGINNER GOAL	Audit every new Sprint 3 API route and make auth, ownership, validation, pagination and error shapes consistent so that the Command Centre and new workflows expose only permitted data and fail predictably.
USER ACCEPTANCE	Given: the combined Sprint 3 API receives valid, invalid, wrong-role and cross-user requests When: the contract/security suite runs Then: allowed calls work and denied calls expose no private data; errors keep the established safe code/message shape
ACCEPTANCE TESTS	- Create endpoint-to-role/ownership matrix for notifications/search/requirements/bulk/reports/import/corrections/overflow. - Search/report responses are paginated/bounded and all write inputs have size limits. - No route trusts browser-supplied ownership where token/profile ownership should decide.
UNIT TESTS + TASKS	Unit: shared validators enforce page/size/version boundaries Unit: error sanitiser keeps code/message and removes stack/private fields [] Audit and fix new endpoint middleware/validators without changing feature meaning. [] Add 400/401/403/404/409/cross-user tests for gaps. [] Update route examples/API evidence for the report/documentation site.

S3-M5-2 | Transaction, idempotency and race hardening

BEGINNER GOAL	Add transaction/idempotency/race protection where repeated or competing writes can occur so that bulk allocation, import, reminders, overflow and corrections never leave duplicate or half-saved state.
USER ACCEPTANCE	Given: bulk allocation, timetable import, notification/reminder, overflow or correction actions are retried/compete When: requests run concurrently or are repeated Then: the database ends in one legal state, duplicates are controlled and failures do not leave partial critical writes
ACCEPTANCE TESTS	- Bulk commit/import use appropriate transaction boundaries and return item-level failures safely. - Retry-prone write endpoints use a concrete idempotency/version strategy where needed. - Notification de-duplication and overflow/claim uniqueness remain race-safe.
UNIT TESTS + TASKS	Unit: idempotency key normaliser rejects malformed/reused incompatible keys Unit: state transition helper rejects stale/illegal transitions [] Run simultaneous-write tests for the new Sprint 3 mutation families. [] Add transaction rollback tests around multi-row operations. [] Do not add idempotency everywhere - only where a repeated write can create duplicate state.

S3-M5-3 | Performance, limits and safe observability

BEGINNER GOAL	Measure and harden the Sprint 3 API endpoints changed in this sprint so that search, reports, bulk actions, import and overflow remain responsive and safely bounded on realistic data.
USER ACCEPTANCE	Given: representative Sprint 3 API requests and malformed/oversized inputs When: Member 5 runs focused timing/limit tests on the APIs they changed Then: high-impact N+1/unbounded cases are fixed, request limits are safe and logs expose no secrets.
ACCEPTANCE TESTS	- Ranking/search/report queries are bounded and avoid obvious per-row query loops. - Safe timing/correlation logging may be added without tokens/private bodies. - Member 5 records focused API limits/evidence; the Integration Lead performs the combined-main release gate.
UNIT TESTS + TASKS	Unit: representative query/service helpers stay deterministic Unit: limit helpers reject oversized arrays/search strings/import files [] Measure the Sprint 3 endpoints Member 5 owns and fix high-impact bottlenecks. [] Run focused API regression for changed routes. [] Hand performance/security evidence to the Integration Lead.

ONE-BRANCH HANDOFF

Keep all Member 5 Sprint 3 work on s3/m5/completion. Self-test only the work on this branch, push it and hand evidence to the Integration Lead. Do not merge to main. The Integration Lead alone reviews/tests the handover, merges it to main and checks updated main. Only then does Member 6 pull that main and create s3/m6/completion.

Sprint 3 - Member 6

BEGINNER-FRIENDLY SUMMARY: Your goal is to build the screen that makes Tutor Mx feel like a real operations product. Use the live staffing, budget, workload, timesheet and overflow data already added by Members 1-5 to create the Organiser Command Centre: clear headline numbers, a shortage heatmap, budget/workload/fairness visuals and a Needs Attention feed that takes the Organiser straight to the problem. You build and self-test this feature only; the Integration Lead handles merging, combined regression and deployment.

STARTING POINT / DO NOT REPEAT

Start from main after Member 5 is merged. Reuse the live Notification, StaffingRequirement, ranking, report/budget, timesheet, excuse and overflow data already integrated. Member 6 is now a visible feature owner, not the release/integration person. Do not duplicate earlier calculations in React; add one aggregate endpoint/service only where the Command Centre needs a combined payload.

S3-M6-1 | Live Command Centre summary + drill-downs

BEGINNER GOAL	Build the Organiser Command Centre home with live headline cards so that an Organiser can understand the staffing situation in seconds instead of opening several screens.
USER ACCEPTANCE	Given: live Sprint 3 staffing/report/workflow data exists When: an Organiser opens the Command Centre and chooses a week/period Then: allocated / required sessions, unfilled work, spend / budget and outstanding-action counts load from the backend, and each card drills into the relevant real screen.
ACCEPTANCE TESTS	• No Command Centre number is hard-coded. • Headline totals use the same definitions as the requirement/report APIs. • Changing week/period refreshes all dependent cards consistently. • Clicking a KPI opens the matching allocation/report/approval/overflow context.
UNIT TESTS + TASKS	Unit: summary mapper handles zero/partial/full staffing Unit: period filter maps the same week to all summary sections [] Build the Command Centre route/layout and reusable KPI cards. [] Add a small aggregate endpoint/service only if multiple existing calls cannot safely provide the summary. [] Add loading/empty/error/retry states and drill-down links. [] Test week changes, zero data, over-budget and partial staffing.

S3-M6-2 | Shortage heatmap + budget/workload/fairness visualisations

BEGINNER GOAL	Turn the live shortage, budget and workload data into understandable visual planning tools so that the Organiser can immediately see which courses are risky, where money is tight and how tutoring work is distributed.
USER ACCEPTANCE	Given: requirement shortages and report/budget/workload data are available When: the Organiser views Command Centre visuals or selects a course Then: the heatmap/gauges/workload view show the same backend values in a visual form, with labels and drill-downs that explain every status.
ACCEPTANCE TESTS	• Shortage heatmap clearly distinguishes fully staffed, partially staffed and unstaffed work with written values as well as colour. • Budget gauge shows spend / budget and over-budget state using backend totals. • Workload/fairness view shows assigned/remaining hours across Tutors without claiming that a soft fairness score is a hard rule. • Selecting a visual item opens the relevant course/Tutor detail.
UNIT TESTS + TASKS	Unit: heatmap status mapper covers zero/partial/full shortage Unit: gauge/fairness formatter handles zero budget, over-budget and no Tutor data [] Build shortage heatmap with accessible text/table fallback. [] Add budget gauge and workload-distribution/fairness view. [] Add course/Tutor drill-down interactions. [] Test values against the raw report/requirement responses.

S3-M6-3 | Needs Attention feed + quick actions

BEGINNER GOAL	Build a Needs Attention feed with direct safe actions so that the Command Centre tells the Organiser what needs action next instead of only showing passive charts.
USER ACCEPTANCE	Given: there are shortages, pending approvals, overdue/returned timesheets, near-capacity Tutors or budget warnings When: the Organiser opens or filters Needs Attention Then: each issue appears once with a reason, urgency/context and a safe link to the exact screen where it can be handled.
ACCEPTANCE TESTS	• Feed can surface unfilled staffing, pending volunteer/excuse/timesheet actions, returned/overdue workflow items, near-capacity Tutors and budget warnings where data exists. • Items are de-duplicated and role-safe; clicking rechecks permission/state. • Quick actions deep-link to existing real workflows rather than reimplementing business rules in the dashboard.
UNIT TESTS + TASKS	Unit: attention-item builder returns stable priority/category IDs Unit: deep-link mapper rejects stale/forbidden targets [] Build filtered Needs Attention list/cards. [] Reuse Member 1 notifications/reminders where appropriate instead of creating duplicate alert storage. [] Add quick links to shortages, approvals, timesheets, Tutors and budget/report context. [] Test empty, stale, duplicate and mixed-category feeds.

ONE-BRANCH HANDOFF

Keep all Member 6 Sprint 3 feature work on s3/m6/completion. Member 6 self-tests only the Command Centre cards/visuals/feed they changed, pushes the branch and hands evidence to the Integration Lead. The Integration Lead alone reviews, merges Member 6 to main, runs the combined Sprint 3 regression/database checks and performs any deployment/release work.

Sprint 3: Integration Lead-only integration and release gate

This page belongs to the Integration Lead - not Member 6 and not any other Member

INTEGRATION LEAD ONLY - MERGE IN THIS EXACT ORDER

After each Member self-tests and hands over their one branch, the Integration Lead reviews it, merges it to main, resolves any merge/integration conflict and checks updated main before allowing the next Member to start. Sequence: M1 -> merge/check -> M2 -> merge/check -> M3 -> M4 -> M5 -> M6. Members never merge themselves and no Member owns the combined-main release gate.

EXISTING BEHAVIOUR THAT MUST STILL WORK

- [] Auth0 role onboarding, protected routes, account lifecycle and one saved role.
- [] Tutor-owned weekly capacity/schedule, WEEKLY/ONCE recurrence, repeat-until and weekly calendar.
- [] Organiser tutor search/workload week navigation and allocation CRUD.
- [] Hard mark + clash + selected-week capacity rules, clashDetails, failed reasons and self-conflict exclusion.
- [] Completed-session work-log guard, current timesheet flow, excuse flow, Student volunteer outcomes/withdraw backend and public-holiday fallback.

NEW SPRINT 3 BEHAVIOUR THAT MUST BE DEMONSTRABLE

- [] Notifications + due reminders; secure global course/person/timesheet search; guided Tutor setup.
- [] Command Centre foundation: live allocated/required + unfilled headline, explainable ranking, shortage heatmap and bulk preview/commit.
- [] Command Centre analytics: course hours, spend/budget and workload distribution reconcile with backend data; Organiser excuse review works.
- [] CSV/ICS/paste timetable import into existing TimeSlot calendar with academic-term bounds.
- [] Returned-timesheet corrections, declaration, disputes/revision history and approved payroll export.
- [] Student withdrawal UI; Organiser overflow post/edit/close; requirement-to-overflow flow.
- [] Keyboard/mobile/representative-user evidence; hardened API transaction/security/performance evidence.

SPRINT 3 DONE MEANS - CONFIRMED BY THE INTEGRATION LEAD

Final main contains approved Members 1-6 in order, the Intermediate features work together end-to-end, current Basic behaviour still works and the public deployment (if released) comes only from approved main. The Integration Lead records the combined regression/CI/database/deployment evidence separately from Member 6 feature work.

Sprint 4: audited plan, order and handovers

30 September-11 October 2026 | Advanced collaboration, explainable allocation and final release

SPRINT 4 MAIN IDEA

Start from approved Sprint 3 main. Advanced work must reuse the same hard eligibility rules and the same live Allocation data. Add safe drafts, live collaboration/conflict handling, Tutor swaps, complete audit/restore and a whole-school explainable proposal - then release-harden everything.

EXACT ORDER

Member	Only new Sprint 4 work
Member 1	Session/auth edge hardening; stale search/notification target safety; actionable notification deep-links and command-palette recent/favourite shortcuts. Focused Member 1 testing only.
Member 2	Draft scenarios side-by-side; assignment locks; Compare with Live/Publish; optimistic concurrency; visible Organiser presence; scenario library management. Focused Member 2 testing only.
Member 3	Tutor swap request, replacement-Tutor accept/decline, Organiser approval, calendar impact/history and safe cancellation using the same hard allocation rules.
Member 4	Race/conflict UI polish plus responsive/accessibility work for dense Command Centre/scenario/audit/proposal screens, including accessible table alternatives for visualisations.
Member 5	Cross-system audit event history, polished timeline, restore preview/restore, and audit filters/export/change-detail tools. Focused Member 5 tests only.
Member 6	Whole-school explainable proposal engine, Why this Tutor?/Why not? cockpit, objective presets and side-by-side strategy comparison/impact lab. Member 6 does not release or deploy.

ADVANCED CONSISTENCY RULES

- Draft scenarios never alter live allocations until explicit Publish.
- Concurrent stale saves never silently overwrite newer data.
- Mark, timetable/clash and selected-week capacity are hard constraints everywhere.
- Locks protect organiser-chosen assignments from proposals/publish until explicitly unlocked.
- Every important mutation is audited; restore creates a new change rather than deleting history.
- Final Command Centre action path: Generate Allocation Proposal -> Why these Tutors? -> Save as Scenario -> Compare with Live -> Publish. Every step reuses the same hard rules, locks, scenario data and audit trail.

Sprint 4 - Member 1

BEGINNER-FRIENDLY SUMMARY: Your goal is to make search, notifications and login/session behaviour feel safe and convenient in the finished product. Harden expired/stale states, then add useful notification deep-links and recent/favourite shortcuts to Ctrl/Cmd+K so users can jump back to common work. Test only Member 1 changes and hand them to the Integration Lead.

STARTING POINT / DO NOT REPEAT

Start from approved Sprint 3 main. Do not rebuild Auth0, the notification inbox, global search or guided Tutor setup. Member 1 improves those existing features only; the Integration Lead owns the later cross-system security/release verification.

S4-M1-1 | Production account and session hardening

BEGINNER GOAL	Harden expired/revoked/invalid session behaviour so that protected Command Centre/search/notification actions stop safely and stale browser state cannot keep using restricted data.
USER ACCEPTANCE	Given: a signed-in session becomes expired, revoked or invalid When: the user tries a protected action, sign-out, reset or delete Then: protected access stops, stale app state is cleared and the correct re-authentication/recovery path is shown
ACCEPTANCE TESTS	• 401/403 meaning stays consistent and no cached protected screen can continue mutating data. • Reset/delete/sign-out clear only intended session/app state. • Network/auth failure does not create a false half-signed-in or half-deleted state.
UNIT TESTS + TASKS	Unit: session-state mapper chooses re-auth vs forbidden correctly Unit: storage cleanup preserves only intended non-sensitive preferences ☐ Audit browser storage/token/session edges in production-like flow. ☐ Fix redirect/retry edge cases without replacing Auth0. ☐ Add expired/revoked/network regression.

S4-M1-2 | Final search/notification stale-target hardening

BEGINNER GOAL	Harden stale/deleted search and notification targets so that old links never reveal private data and users get a safe No longer available state instead of broken screens.
USER ACCEPTANCE	Given: a saved notification or previous search result points to a record that was deleted or is no longer allowed When: the user follows the link or searches again Then: no private data is shown; a safe No longer available/Not found state appears and fresh search omits the record
ACCEPTANCE TESTS	• Permission is rechecked on target open, not trusted from old result data. • Read/unread and reminder de-dup still work with stale targets. • Search cache/result refresh cannot retain newly forbidden records.
UNIT TESTS + TASKS	Unit: stale-target helper maps missing/forbidden without leaking details Unit: permission filter reflects role/ownership change ☐ Run production-like stale/deleted/restricted cases. ☐ Fix only release defects; keep one search and one inbox system. ☐ Add regression for changed ownership/visibility.

S4-M1-3 | Actionable notifications + command-palette recent/favourites

BEGINNER GOAL	Add safe notification actions plus recent/favourite destinations in Ctrl/Cmd+K so that users can return to common Tutor Mx work quickly without weakening role/ownership checks.
USER ACCEPTANCE	Given: a user has permitted notifications/search destinations and recent activity When: they open a notification action or Ctrl/Cmd+K recent/favourites Then: the correct current destination opens only after fresh permission/state checks; stale or forbidden targets show a safe unavailable state.
ACCEPTANCE TESTS	• Notification cards may include safe View/Fix/Review links to existing routes. • Ctrl/Cmd+K can show recent/favourite permitted destinations without exposing another role/user. • Removed/restricted targets disappear or show safe unavailable state after recheck.
UNIT TESTS + TASKS	Unit: recent/favourite dedupe/order is deterministic Unit: action-link permission mapper rejects stale/forbidden targets ☐ Add safe notification action links. ☐ Add recent/favourite entries to the existing command palette. ☐ Test stale, deleted, wrong-role and keyboard flows on Member 1 branch.

ONE-BRANCH HANDOFF

Keep all Member 1 Sprint 4 work on s4/m1/completion. Self-test only the work on this branch, push it and hand evidence to the Integration Lead. Do not merge to main. The Integration Lead alone reviews/tests the handover, merges it to main and checks updated main. Only then does Member 2 pull that main and create s4/m2/completion.

Sprint 4 - Member 2

BEGINNER-FRIENDLY SUMMARY: Your goal is to turn the Command Centre into a safe planning workspace where Organisers can create alternatives without touching live allocations. Build scenarios, locks, Compare with Live, Publish and live presence/conflict handling, then add a simple scenario library so drafts can be named, copied and archived. The Integration Lead will later test the combined system.

STARTING POINT / DO NOT REPEAT

Start from main after Member 1 is merged. Reuse Sprint 3 requirements/ranking/shortages/bulk/reports and normal allocation CRUD. Any Scenario/version/lock/presence storage and API needed here is owned by this Member 2 branch.

S4-M2-1 | Command Centre scenarios: side-by-side compare, locks and publish

BEGINNER GOAL	Build draft allocation scenarios, assignment locks, side-by-side compare and explicit publish inside the Organiser Command Centre so that Organisers can experiment with alternatives without touching live allocations until they deliberately publish.
USER ACCEPTANCE	Given: an Organiser wants to test allocation alternatives When: they create/edit drafts, compare them, lock selected assignments and choose Publish Then: draft changes remain isolated until Publish; differences/shortages/workload are visible and locked assignments stay unchanged
ACCEPTANCE TESTS	- The Command Centre presents Draft vs Live clearly and exposes Save as Scenario / Compare with Live / Publish only when each action is valid. - Scenario stores draft assignment choices separately from live Allocation rows. - Compare labels added/removed/changed assignment, shortage, workload and budget differences. - Publish revalidates hard rules and is all-or-nothing for the scenario.
UNIT TESTS + TASKS	Unit: scenario diff helper is deterministic Unit: lock/publish validator preserves locked assignments and hard-rule failures [] Add Scenario/ScenarioItem/lock fields or minimal equivalent + API/UI on this branch. [] Build side-by-side diff and explicit Publish confirmation. [] Test delete/rename/compare/publish failure and locked assignment cases.

S4-M2-2 | Concurrent Organiser conflict handling + visible presence

BEGINNER GOAL	Add optimistic concurrency and visible Organiser presence so that two Organisers can work on the same board, see who else is active, and get a clear conflict instead of silently overwriting each other.
USER ACCEPTANCE	Given: two Organisers open the same board/item When: one saves while the other still holds an older version Then: the second stale save is rejected; both can see active collaborators and refresh/compare/retry without losing selected context
ACCEPTANCE TESTS	- Version/updatedAt expected value is required on versioned writes; stale save returns conflict. - Presence strip/avatar list shows active Organisers on the current board/item with a short heartbeat/TTL; it is not permission authority. - Conflict recovery preserves selected week/scenario/filter context.
UNIT TESTS + TASKS	Unit: version comparator rejects stale expected version Unit: presence TTL helper expires inactive sessions predictably [] Add optimistic version fields and presence heartbeat storage/route/UI on this branch. [] Show last updated/by and conflict compare/retry path. [] Test in two browser sessions including one abrupt disconnect/expiry.

S4-M2-3 | Scenario library: rename, duplicate, notes and archive

BEGINNER GOAL	Build a simple scenario library so that Organisers can organise multiple draft plans, duplicate a useful draft, add notes and archive old alternatives without confusing them with live allocations.
USER ACCEPTANCE	Given: several draft scenarios exist When: an Organiser renames, duplicates, notes, filters or archives them Then: live allocations stay unchanged, each draft keeps its own identity/version and the Organiser can reopen the correct scenario from the Command Centre.
ACCEPTANCE TESTS	- Duplicate creates a separate draft copy, never a live allocation. - Archive hides a draft from the default list without deleting audit/history references. - Name/note validation and stale-version conflicts are handled safely.
UNIT TESTS + TASKS	Unit: scenario clone mapper creates new IDs while preserving intended items Unit: archive/filter helper returns only requested states [] Add scenario list/library UI with rename, duplicate, notes and archive. [] Reuse optimistic version checks from S4-M2-2. [] Test duplicate/archive/reopen/stale rename and empty-library states.

ONE-BRANCH HANDOFF

Keep all Member 2 Sprint 4 work on s4/m2/completion. Self-test only the work on this branch, push it and hand evidence to the Integration Lead. Do not merge to main. The Integration Lead alone reviews/tests the handover, merges it to main and checks updated main. Only then does Member 3 pull that main and create s4/m3/completion.

Sprint 4 - Member 3

BEGINNER-FRIENDLY SUMMARY: Your goal is to make Tutor session trading work like a real workflow. A Tutor proposes a future-session swap, the replacement Tutor can accept or decline, and only then can an Organiser approve it. Show the calendar/workload impact before final approval and keep a clear swap history so nobody is surprised by a changed session.

STARTING POINT / DO NOT REPEAT

Start from main after Member 2 is merged. Do not rebuild Tutor capacity, schedule/import, timesheet correction/dispute/declaration or payroll export. Build swaps as a new workflow using the same hard allocation rules.

S4-M3-1 | Tutor swap request + eligible replacement check

BEGINNER GOAL	Build the first half of the swap workflow so that a Tutor can propose a future session trade only to a replacement Tutor who passes the exact same mark, clash and selected-week capacity rules.
USER ACCEPTANCE	Given: a Tutor selects one of their confirmed future sessions When: they search/select a possible replacement and submit a swap request Then: ineligible replacements are blocked with the existing reasons and a valid request enters PENDING_REPLACEMENT without changing the live allocation.
ACCEPTANCE TESTS	- Replacement eligibility reuses the normal hard-rule service. - Creating a swap request does not change the live Allocation. - Only the original Tutor can create/cancel their pending request; duplicate requests are controlled.
UNIT TESTS + TASKS	Unit: swap candidate validator maps existing reason codes Unit: create/cancel state helper rejects illegal transitions [] Add Swap request/status storage + proposal UI/API. [] Reuse existing rule-result components and notifications. [] Test valid, rule-fail, duplicate, stale and wrong-user request creation.

S4-M3-2 | Replacement Tutor accept/decline + Organiser approval

BEGINNER GOAL	Add replacement-Tutor accept/decline and Organiser approve/reject so that a session trade requires agreement from the people involved and a final hard-rule recheck before the live allocation changes.
USER ACCEPTANCE	Given: a valid swap request is waiting for the replacement Tutor When: the replacement accepts/declines and, if accepted, the Organiser reviews it Then: decline ends the request without changing live data; Organiser approval rechecks rules/version and atomically changes the assignment only if still valid.
ACCEPTANCE TESTS	- Replacement Tutor sees only swap requests addressed to them. - Organiser cannot approve before replacement acceptance. - Approval rechecks mark/clash/capacity and version; stale or newly invalid swaps change nothing. - All affected users get the correct notification.
UNIT TESTS + TASKS	Unit: swap transition helper enforces request -> replacement decision -> organiser decision Unit: atomic apply helper changes assignment only on approved valid state [] Add replacement accept/decline UI/API. [] Add Organiser review/approve/reject UI/API. [] Test rejection, stale approval, changed capacity/clash and concurrent allocation change.

S4-M3-3 | Swap calendar impact, history and safe cancellation

BEGINNER GOAL	Show swap before/after calendar impact and a clear history so that original/replacement Tutors can understand what will change, review past outcomes and safely cancel only while cancellation is still legal.
USER ACCEPTANCE	Given: a pending or completed swap exists When: a Tutor opens swap detail/history or cancels an allowed pending request Then: before/after session dates, weekly-hour impact, status timeline and permitted actions are clear, while completed/rejected swaps stay immutable.
ACCEPTANCE TESTS	- Preview shows original vs proposed session/Tutor and selected-week hours. - History shows requested/accepted/declined/approved/rejected/cancelled timestamps and actors. - Cancel is available only before the defined lock point and never rewrites completed history.
UNIT TESTS + TASKS	Unit: calendar-impact helper returns before/after selected-week hours Unit: cancellation rule allows only permitted pending states [] Add swap detail/history timeline and before/after preview. [] Add safe cancellation where allowed. [] Test history visibility, locked cancellation and calendar update after approval.

ONE-BRANCH HANDOFF

Keep all Member 3 Sprint 4 work on s4/m3/completion. Self-test only the work on this branch, push it and hand evidence to the Integration Lead. Do not merge to main. The Integration Lead alone reviews/tests the handover, merges it to main and checks updated main. Only then does Member 4 pull that main and create s4/m4/completion.

Sprint 4 - Member 4

BEGINNER-FRIENDLY SUMMARY: Your goal is to make the Advanced screens understandable when they get dense. Polish conflict messages, make the Command Centre/scenario/audit/proposal UI work on phone/tablet/desktop and keyboard, and provide accessible table/list alternatives for heatmaps/charts so the visual information is never available only by colour or graphics.

STARTING POINT / DO NOT REPEAT

Start from main after Member 3 is merged. Do not redesign working volunteer/withdrawal/overflow or shared components unless a real release issue is reproduced.

S4-M4-1 | Two-session race/conflict and notification polish

BEGINNER GOAL	Test and polish two-session conflict/race states so that competing Student/Organiser actions converge on one server truth and the UI clearly tells users to refresh/retry instead of showing impossible states.
USER ACCEPTANCE	Given: two sessions act on the same overflow/claim or shared action nearly together When: claim/withdraw/approve/close actions compete Then: one legal final state wins, stale action gets conflict + refresh/retry, and affected users receive only relevant non-duplicate notifications
ACCEPTANCE TESTS	- Competing Student/Organiser actions converge on one persisted state. - UI refreshes to server truth without leaving an impossible button/status combination. - Notification display does not duplicate the same event key/outcome.
UNIT TESTS + TASKS	Unit: conflict response mapper chooses refresh/retry state Unit: notification display de-dup keeps distinct outcomes [] Run two-session overflow race scripts. [] Polish conflict banner/dialog and refresh path. [] Fix duplicate/irrelevant overflow notifications or UI state only.

S4-M4-2 | Final responsive/accessibility visual polish

BEGINNER GOAL	Polish the final Command Centre and all role screens for phone/tablet/desktop and keyboard use so that dense analytics, scenario comparison, proposal explanations and audit history remain readable and accessible.
USER ACCEPTANCE	Given: final Student/Tutor/Organiser screens contain all components When: they are used on common widths and with keyboard/assistive semantics Then: content is readable, whole pages do not scroll sideways, focus/labels/status announcements are correct and dense planning screens remain understandable
ACCEPTANCE TESTS	- Calendars, allocation studio, reports, import previews, scenario diff and proposal-ready shared components do not clip primary actions. - Focus restoration works after dialogs/conflicts and live/status announcements use text, not colour alone. - Contrast/labels/headings remain coherent across roles.
UNIT TESTS + TASKS	Unit: shared components cover long text/narrow width/focus restore Unit: live-region helper maps success/error/conflict message correctly [] Audit navigation, cards, calendars, dialogs, tables and forms. [] Run contrast/focus/label/status checks and capture common-width evidence. [] Apply visual consistency fixes only where they do not alter business rules.

S4-M4-3 | Accessible data-table alternatives + compact planning mode

BEGINNER GOAL	Add accessible table/list alternatives and a compact responsive planning mode so that shortage, budget, workload and proposal information remains understandable on small screens and without relying on colour/graphics.
USER ACCEPTANCE	Given: Command Centre heatmaps/charts/scenario/proposal summaries contain dense data When: a user switches to table/list view or opens the page on a narrow screen Then: the same values, labels, status meanings and actions remain available in a readable keyboard-accessible format.
ACCEPTANCE TESTS	- Every visual has an equivalent labelled data view for its important values. - Compact mode prioritises primary action/status without hiding required information. - Colour is never the only status signal; focus order and live messages remain correct.
UNIT TESTS + TASKS	Unit: visual-to-table mapper preserves labels/values/status Unit: compact-view helper returns the same underlying data [] Add View as table/list to key heatmap/chart components. [] Add compact responsive layout for dense planning screens. [] Test keyboard/focus/long-text/narrow-width behaviour on Member 4 changes.

ONE-BRANCH HANDOFF

Keep all Member 4 Sprint 4 work on s4/m4/completion. Self-test only the work on this branch, push it and hand evidence to the Integration Lead. Do not merge to main. The Integration Lead alone reviews/tests the handover, merges it to main and checks updated main. Only then does Member 5 pull that main and create s4/m5/completion.

Sprint 4 - Member 5

BEGINNER-FRIENDLY SUMMARY: Your goal is to make important changes easy to investigate and safely undo. Build one audit trail and timeline, add restore preview/restore, then make the history genuinely useful with filters, before/after detail and a safe export. You test these audit features; the Integration Lead owns the final whole-system security/release checks.

STARTING POINT / DO NOT REPEAT

Start from main after Member 4 is merged. Scenario, presence and swap routes already exist. Member 5 adds one shared audit/version capability, restore UI/API and usable audit investigation tools. Member 5 does not own the final integrated API/release gate.

S4-M5-1 | Cross-system audit trail + polished Command Centre timeline

BEGINNER GOAL	Build one cross-system audit service and polished Organiser audit timeline so that every important successful change answers who changed what, when and why without exposing sensitive data.
USER ACCEPTANCE	Given: an important protected mutation succeeds When: the Organiser opens audit history for the relevant record/context Then: actor, time, action, target and safe before/after references are visible without exposing secrets/private token data
ACCEPTANCE TESTS	- The audit timeline is reachable from the Command Centre and can filter to the selected course/allocation/scenario context. - Audit is created only for committed successful mutations and uses authenticated actor identity. - Important mutation families are covered by shared audit service/pattern, not copy-paste ad hoc logs. - Reason fields from manual choice/return/reject/restore are retained where relevant.
UNIT TESTS + TASKS	Unit: audit record builder requires actor/action/target metadata Unit: redaction helper removes sensitive fields from before/after snapshot [] Add AuditEvent/minimal snapshot schema, shared service/hooks and Organiser audit timeline on this branch. [] Instrument existing important mutation routes after successful transaction. [] Test rollback/no-audit, actor attribution, filters and permission.

S4-M5-2 | Allocation restore preview -> restore as a new auditable change

BEGINNER GOAL	Build allocation restore preview and safe restore-as-new-change so that an Organiser can compare current vs historical state and recover a valid earlier allocation without deleting history.
USER ACCEPTANCE	Given: an allocation has audited previous versions When: the Organiser previews a prior state and confirms Restore Then: hard rules/version are rechecked; a valid restore creates a new current state and new audit event while the original history remains intact
ACCEPTANCE TESTS	- Restore preview clearly shows current vs target values and affected Tutor/course/time. - Restore refuses stale/invalid/hard-rule-breaking target state. - Successful restore is a new mutation/event; no historic row/event is erased.
UNIT TESTS + TASKS	Unit: restore validator rechecks version + hard eligibility Unit: diff helper highlights changed assignment/time/status fields [] Add history/preview/restore endpoint + UI on this branch. [] Use transactions and expected version to prevent stale restore. [] Test restore success, hard-rule fail, stale version, permission and rollback.

S4-M5-3 | Audit search/filter, change detail + safe export

BEGINNER GOAL	Add useful audit search/filter, before/after change detail and safe export so that an Organiser can investigate a course, Tutor, allocation, scenario or date range without manually reading raw database records.
USER ACCEPTANCE	Given: many AuditEvents exist across different mutation types When: an authorised Organiser filters/searches, opens a change or exports the selected history Then: matching events are bounded/paginated, sensitive fields stay redacted and the displayed/exported before/after detail matches the stored audit record.
ACCEPTANCE TESTS	- Filter by relevant date, actor, action and target/context. - Change detail shows safe before/after fields and reason where recorded. - Export contains only authorised/redacted audit fields and the selected filter scope. - Large history queries are bounded/paginated.
UNIT TESTS + TASKS	Unit: audit filter builder applies actor/action/target/date boundaries correctly Unit: export formatter keeps redaction and deterministic columns [] Add audit filters/search and change-detail drawer. [] Add safe CSV export for authorised filtered history. [] Test permissions, pagination, no-results, redaction and export escaping.

ONE-BRANCH HANDOFF

Keep all Member 5 Sprint 4 work on s4/m5/completion. Self-test only the work on this branch, push it and hand evidence to the Integration Lead. Do not merge to main. The Integration Lead alone reviews/tests the handover, merges it to main and checks updated main. Only then does Member 6 pull that main and create s4/m6/completion.

Sprint 4 - Member 6

BEGINNER-FRIENDLY SUMMARY: Your goal is to build the final intelligence layer of Tutor Mx. Generate an explainable whole-school draft allocation, show Why this Tutor? / Why not?, and then let the Organiser compare different strategies such as Balanced, Strongest Match, Fair Workload and Budget Aware before choosing one to save as a Scenario. You self-test the proposal features only; the Integration Lead handles merge, final regression and deployment.

STARTING POINT / DO NOT REPEAT

Start from main after Member 5 is merged. Reuse StaffingRequirement, hard eligibility, ranking factors, scenarios, locks, budgets, audit and live Allocation. Member 6 owns proposal intelligence only. The proposal engine creates drafts/scenarios and never deploys or writes directly to live allocations.

S4-M6-1 | Whole-school explainable proposal engine with soft-objective presets

BEGINNER GOAL	Build the whole-school proposal engine with soft-objective presets so that Tutor Mx can propose a complete draft allocation across open requirements while always respecting hard rules and Organiser locks.
USER ACCEPTANCE	Given: school-wide open requirements, eligible Tutors, budgets/workload and locked assignments exist When: the Organiser selects a soft-objective preset and generates a proposal Then: the same input/preset gives the same draft plan; hard rules/locks are never broken; remaining shortages and objective trade-offs are returned
ACCEPTANCE TESTS	- Hard constraints: mark, clash and selected-week capacity. Locks remain untouched. - Soft objectives can balance shortage first, stronger marks, fairer workload and budget/spend; presets change weights only. - Proposal output is a normal draft Scenario, never live data.
UNIT TESTS + TASKS	Unit: optimizer never selects isEligible=false or changes locked item Unit: objective scoring/tie-breaker deterministic on repeated fixtures [] Build proposal engine on this branch using existing eligible/ranking/requirement data. [] Add presets such as Balanced, Strongest Match, Fair Workload and Budget Aware with documented weights. [] Test no solution, tie, lock, budget pressure, week boundary and deterministic repeat.

S4-M6-2 | Command Centre proposal cockpit: Why this Tutor? / Why not?

BEGINNER GOAL	Add the final Command Centre proposal cockpit with Generate Allocation Proposal, Why these Tutors?, shortage map and compare/edit links so that the Organiser can understand every suggestion, save it as a Scenario, compare with Live and publish only after review.
USER ACCEPTANCE	Given: a generated proposal draft exists When: the Organiser opens a requirement/assignment explanation and compares the proposal with live/another scenario Then: the UI shows chosen factors, skipped reasons, hard-rule failures, soft-score contribution, locks, shortages and scenario differences in plain language
ACCEPTANCE TESTS	- The final Command Centre presents the sequence Generate Allocation Proposal -> Why these Tutors? -> Save as Scenario -> Compare with Live -> Publish using live data and the existing scenario workflow. "Why this Tutor?" lists hard-rule pass + soft factors; "Why not?" distinguishes hard failure from lower soft score. - Shortage heatmap and budget/workload impact use the same report definitions as Sprint 3. - Organiser can manually edit the draft to another eligible Tutor with reason before normal Publish.
UNIT TESTS + TASKS	Unit: explanation formatter is stable and never claims a factor not in engine input Unit: proposal-to-scenario mapper preserves locks/shortages/reasons [] Build proposal cockpit UI, explanation drawer and compare link on this branch. [] Reuse scenario diff, audit and ranking components rather than duplicate them. [] Test explanation accuracy, keyboard navigation and manual eligible adjustment.

S4-M6-3 | Strategy comparison + proposal impact lab

BEGINNER GOAL	Build a strategy-comparison and proposal-impact lab so that the Organiser can run multiple soft-objective presets and compare their consequences before deciding which draft is worth saving/publishing.
USER ACCEPTANCE	Given: the same school-wide requirements/tutors/locks are available When: the Organiser generates two or more presets such as Balanced, Strongest Match, Fair Workload or Budget Aware Then: the Command Centre compares coverage, remaining shortage, spend/budget and workload-distribution impact side by side and lets the Organiser open/save one result as a normal Scenario.
ACCEPTANCE TESTS	- Presets change only soft-objective weights; hard eligibility and locks never change. - Side-by-side metrics use the same shortage/budget/workload definitions as Sprint 3. - Same input + same preset stays deterministic. - Choosing a result saves/opens a draft Scenario; it never directly publishes live allocations.
UNIT TESTS + TASKS	Unit: preset-comparison helper uses identical input and returns deterministic metrics Unit: impact mapper reconciles coverage/shortage/spend/workload with existing report definitions [] Add multi-preset comparison UI in the proposal cockpit. [] Show side-by-side impact cards/table and explanation links. [] Add Save chosen proposal as Scenario / Compare with Live handoff to the existing scenario workflow. [] Test ties, no-solution, lock-heavy and budget-pressure cases.

ONE-BRANCH HANDOFF

Keep all Member 6 Sprint 4 proposal/intelligence work on s4/m6/completion. Member 6 self-tests the proposal engine, explanations and strategy comparison, pushes the branch and hands evidence to the Integration Lead. The Integration Lead alone reviews, merges to main, runs the final project regression/security/database checks, syncs the deployment mirror and redeploys/verifies production.

Sprint 4: Integration Lead-only final integration and release gate

This page is performed by the Integration Lead after Member 6 hands over - no Member owns this work

INTEGRATION LEAD ONLY - FINAL SEQUENCE

Member 1 hands over -> Integration Lead reviews/tests/merges -> Member 2 pulls updated main -> repeat through Member 6. After Member 6 is merged, the Integration Lead alone runs the combined Basic + Sprint 3 + Sprint 4 regression, database/security/concurrency checks, confirms all branches landed correctly, syncs approved Gitea main to the GitHub deployment mirror, deploys Render/Cloudflare and records the final public smoke/release SHA.

ADVANCED BEHAVIOUR THAT MUST BE DEMONSTRABLE

- [] Draft allocation scenarios stay separate from live until explicit Publish; side-by-side diff works.
- [] Organiser locks protect chosen assignments; stale concurrent writes return conflict rather than overwrite.
- [] Visible organiser presence shows who else is active on the same board/context.
- [] Tutor swaps recheck the same mark/clash/selected-week capacity constraints and require approval.
- [] Audit timeline records important mutations with actor/time/action; allocation restore creates a new auditable change.
- [] Final Command Centre supports Generate Proposal -> Why these Tutors? -> Save as Scenario -> Compare with Live -> Publish, plus audit timeline/restore, all on the same live data and hard rules.

FINAL REGRESSION AND RELEASE PROOF

- [] Run Basic + Sprint 3 + Sprint 4 automated tests and clean-browser role tests.
- [] Run database/migration/recovery, security, idempotency, concurrency and relevant performance checks.
- [] Confirm every Member 1-6 branch merged in order and no earlier feature disappeared.
- [] Approved Gitea main SHA is synced to GitHub deployment mirror.
- [] Render backend and fresh Cloudflare frontend are deployed from that same approved source.
- [] Run public Student/Tutor/Organiser smoke and record release SHA/evidence.

SPRINT 4 DONE MEANS - CONFIRMED BY THE INTEGRATION LEAD

Approved final main contains all six Member feature branches in order; every Basic, Intermediate and Advanced feature can be demonstrated; hard rules remain consistent; no earlier behaviour regressed; and the deployed frontend/backend correspond to the same approved source. This combined-main/release proof is Integration Lead work, not a Member 6 card.

Final requirement traceability matrix

Project brief -> current main foundation -> new work that closes the gap

Brief requirement	Current main	Final owner
Courses/tutors/marks/hours/busy times	Already in main	Regression only
Single allocation board; live mark/clash/hours validation	Already in main	Regression only
Work logs/timesheets/Organiser sign-off	Mostly in main	S3 M2 excuse review UI; S3 M3 declaration/corrections/disputes/export
Tutor excuse; Student volunteer	In main; excuse decision UI gap; Student withdrawal UI gap	S3 M2 + M4
Rank eligible Tutors	Primitive mark ordering exists	S3 M2 explicit multi-factor explanation
Bulk obvious assignments + shortages + reasoned override	Missing	S3 M2
Import timetable + recurring exceptions + term calendar	Manual recurrence exists; import/term missing	S3 M3
Due prompts + assignment/status notifications	Missing	S3 M1
Overflow posted/claimed/approved	Claim/approval exists; posting UI/API missing	S3 M4
Hours/course, spend/budget, work spread	Generic per-Tutor workload exists; brief analytics missing	S3 M2 calculations/data + S3 M6 Command Centre visualisation
Guided Tutor setup, global course/person/timesheet search, keyboard/bulk speed	Partial capacity hint/local tutor search	S3 M1 search/setup + S3 M2 bulk + S3 M4 accessibility + S3 M6 Command Centre quick actions
Whole-school allocator balancing objectives + locks + explanations	Missing	S4 M2 locks/scenarios; S4 M6 proposal cockpit
Multiple Organisers see each other; conflicts resolved	Missing	S4 M2 presence + optimistic concurrency
Draft side-by-side scenarios + publish	Missing	S4 M2
Tutor trades/swaps checked + approved	Missing	S4 M3
Every important change recorded; restore allocation	Missing	S4 M5

COVERAGE CHECK

By the end of Sprint 4, every explicit Basic, Intermediate and Advanced requirement in the supplied brief has either an already-working foundation or a named implementation owner and release test in this roadmap.

Stand-out features that add polish without random scope creep

These enhancements directly strengthen a required workflow instead of creating unrelated features

Allocation Command Centre The organiser home combines live staffing progress, unfilled work, shortage heatmap, spend/budget and workload. Sprint 4 layers in scenarios, Why these Tutors?, proposal generation and the audit timeline instead of creating separate disconnected dashboards.

Polish	Why it belongs
Ctrl/Cmd+K command palette	Makes the required global search genuinely quick for a keyboard-heavy organiser.
Shortage heatmap + budget gauges	Turns required shortage/budget/workload data into an instantly understandable planning view.
CSV + .ics import preview	Makes timetable import feel professional and safer than silent bulk upload.
Requirement -> overflow action	Connects allocation shortage directly to the required Student volunteer workflow.
Scenario diff view	Makes side-by-side drafts understandable: added/removed/changed assignment, shortage, budget and workload.
Live organiser presence	Directly satisfies "see where each other are" with active-user chips/avatars plus safe TTL.
Audit timeline + restore preview	Makes history/restore trustworthy by showing exactly what will change before confirmation.
Explainable proposal cockpit	The showcase feature: objective presets, locks, shortage map, "Why this Tutor?" and "Why not?" over the same hard constraints.

SUGGESTED FINAL DEMO FLOW

- Open Tutor Mx Command Centre -> choose week -> show live allocated/required sessions, how many still need Tutors, projected/approved spend vs budget and the course shortage heatmap.
- Open a shortage -> inspect explainable eligible ranking -> bulk preview obvious assignments -> choose a different eligible Tutor with a written reason -> commit and watch Command Centre budget/workload numbers refresh.
- Open two Organiser windows -> show live presence -> create a stale edit -> demonstrate conflict/compare/refresh instead of silent overwrite.
- From the Command Centre create two draft scenarios -> compare each with Live -> lock one assignment -> keep both drafts separate from live data.
- Click Generate Allocation Proposal -> open Why these Tutors? / Why not? -> inspect hard-rule vs soft-objective explanations -> Save as Scenario -> Compare with Live -> Publish the chosen plan.
- Switch to Tutor -> import timetable -> show clash impact -> log completed work -> returned correction/dispute -> approved export -> propose swap.
- Switch to Student -> volunteer -> withdraw pending request -> show organiser overflow posting/approval and notification.
- Return to the Command Centre -> open the audit timeline -> preview a restore -> restore as a new auditable event -> finish by showing the recorded release SHA/evidence.

SCOPE CONTROL
Do not add decorative AI/chat features, gamification or unrelated dashboards just because they look modern. The strongest "cool" features here make the project brief easier to demonstrate and defend technically.

COMMAND CENTRE RULE
The Command Centre is a presentation/action surface over live APIs and shared backend rules - not a second source of truth.

INTEGRATION LEAD RULE
Members build and self-test only their assigned branch work. The Integration Lead alone reviews handovers, merges to main, runs combined-main regression, resolves integration conflicts, synchronises the deployment mirror and deploys/verifies the public system.

Final beginner memory aid

The short version for every team member

- One Member = one branch per sprint. Put every card for that Member on s3/mX/completion or s4/mX/completion.
- Always start from updated main. Member 2 starts only after Member 1 is merged; continue in exact order through Member 6.
- No forward dependencies: if your feature needs a small route/table/migration, include it on your branch. Later Members harden; they do not unblock earlier Members.
- Do not rebuild working features. Current main snapshot 02e8bc3... is the baseline used for this audit.
- User acceptance: Given / When / Then real-user story. Acceptance: whole feature end-to-end. Unit: one helper/component/service by itself.
- Hard allocation rules stay fixed: mark AND timetable/clash AND selected-week capacity must pass. Soft ranking/objectives may change; hard eligibility may not.
- Intermediate completion means the Command Centre foundation is real: allocation progress, shortages, explainable ranking/bulk, budget/workload analytics, search/notifications, timetable import, timesheet corrections/disputes/export and overflow management.
- Advanced completion means the same Command Centre gains scenarios/locks/presence/conflicts, Tutor swaps, audit/restore and the explainable whole-school proposal flow: Generate -> Why -> Save as Scenario -> Compare with Live -> Publish.
- After your branch: run focused local tests/browser acceptance for your own work, push, wait for your branch CI/Codecov and hand over evidence. Do not merge to main or perform the combined release gate. The next Member starts only after the Integration Lead has merged and checked updated main.
- Integration Lead only: review every handover -> merge to Gitea main -> rerun affected/combined checks -> resolve integration conflicts -> sync approved main to GitHub deployment mirror -> deploy Render/backend + fresh Cloudflare frontend -> public three-role smoke -> record release SHA.

SOURCE NOTE

This Sprint 3-4 replacement was produced from a static audit of the supplied current-main source archive (SHA 02e8bc3d1304275550cb7abb89ebc7dee1024b6f), the supplied COMS3011A Tutor Management System project brief, and the existing handbook. Sprint 1-2 and the post-Sprint 2 stabilisation pages remain historical evidence; pages 30 onward replace the prior Sprint 3-4 roadmap.

Key current-main evidence used in the audit: src/app.ts; prisma/schema.prisma; src/services/allocation-queries.ts; frontend/src/features/OrganiserDashboard.jsx; TutorManagement.jsx; TutorDashboard.jsx; AvailabilityForm.jsx; TimesheetSection.jsx; OrganiserApprovals.jsx; StudentOverflow.jsx; frontend/src/api/studentVolunteer.js.

FINAL RULE

Before implementing any roadmap item, verify current code still matches the audited baseline. If a feature was added to main after this PDF was produced, narrow the card to the remaining gap rather than building it twice. For Sprint 3 and 4, all main-branch integration/release/deployment work belongs only to the Integration Lead and is never assigned as a Member 1-6 feature card.